



Module M36

Partha Pratim
Das

Weekly Recap

Objective &
Outline

Exception
Fundamentals

Types of Exceptions
Exception Stages

Error Handling in
C

C Language Features
RV & Params
Local goto
C Standard Library
Support

Global Variables
Abnormal
Termination
Conditional
Termination

Non-Local goto
Signals

Shortcomings

Module Summary

Programming in Modern C++

Module M36: Exceptions (Error handling in C): Part 1

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Weekly Recap

Module M36

Partha Pratim Das

Weekly Recap

Objective & Outline

Exception Fundamentals

Types of Exceptions
Exception Stages

Error Handling in C

C Language Features

RV & Params

Local goto

C Standard Library Support

Global Variables

Abnormal Termination

Conditional Termination

Non-Local goto

Signals

Shortcomings

Module Summary

- Leveraging an innovative solution to the Salary Processing Application in C using function pointers, we compare C and C++ solutions to the problem
- The new C solution with function pointers is used to explain the mechanism for dynamic binding (polymorphic dispatch) based on `virtual` function tables
- Understood casting in C and C++
- Explained cast operators in C++ and discussed the evils of C-style casting
- Studied `const_cast`, `static_cast`, `reinterpret_cast`, and `dynamic_cast` with examples
- Understood casting at run-time with RTTI and `typeid` operator
- Introduced the Semantics of Multiple Inheritance in C++
- Discussed the Diamond Problem and solution approaches
- Illustrated the design choice between inheritance and composition



Module Objectives

- Understand the Error handling in C

NPTEL

Module M36

Partha Pratim
Das

Weekly Recap

Objective &
Outline

Exception
Fundamentals

Types of Exceptions
Exception Stages

Error Handling in
C

C Language Features

RV & Params

Local goto

C Standard Library
Support

Global Variables

Abnormal
Termination

Conditional
Termination

Non-Local goto

Signals

Shortcomings

Module Summary



Module Outline

Module M36

Partha Pratim Das

Weekly Recap

Objective & Outline

Exception Fundamentals

Types of Exceptions

Exception Stages

Error Handling in C

C Language Features

RV & Params

Local goto

C Standard Library Support

Global Variables

Abnormal Termination

Conditional Termination

Non-Local goto

Signals

Shortcomings

Module Summary

- 1 Weekly Recap
- 2 Exception Fundamentals
 - Types of Exceptions
 - Exception Stages
- 3 Error Handling in C
 - C Language Features
 - Return Value & Parameters
 - Local goto
 - C Standard Library Support
 - Global Variables
 - Abnormal Termination
 - Conditional Termination
 - Non-Local goto
 - Signals
 - Shortcomings
- 4 Module Summary



Exception Fundamentals

Module M36

Partha Pratim
Das

Weekly Recap

Objective &
Outline

**Exception
Fundamentals**

Types of Exceptions

Exception Stages

Error Handling in
C

C Language Features

RV & Params

Local goto

C Standard Library
Support

Global Variables

Abnormal
Termination

Conditional
Termination

Non-Local goto

Signals

Shortcomings

Module Summary

Exception Fundamentals



What are Exceptions?

Module M36

Partha Pratim Das

Weekly Recap

Objective & Outline

Exception Fundamentals

Types of Exceptions

Exception Stages

Error Handling in C

C Language Features

RV & Params

Local goto

C Standard Library Support

Global Variables

Abnormal Termination

Conditional Termination

Non-Local goto

Signals

Shortcomings

Module Summary

- Conditions that arise
 - Infrequently and Unexpectedly
 - Generally betray a Program Error
 - Require a considered Programmatic Response
 - Run-time Anomalies – yes, but not necessarily
- Leading to
 - Crippling the Program
 - May pull the entire System down
 - Defensive Technique
 - ▷ Crashing Exceptions verses Tangled Design and Code



Exception Causes

Module M36

Partha Pratim Das

Weekly Recap

Objective & Outline

Exception Fundamentals

Types of Exceptions

Exception Stages

Error Handling in C

C Language Features

RV & Params

Local goto

C Standard Library Support

Global Variables

Abnormal Termination

Conditional Termination

Non-Local goto

Signals

Shortcomings

Module Summary

- Unexpected Systems State
 - Exhaustion of Resources
 - ▷ Low Free Store Memory
 - ▷ Low Disk Space
 - Pushing to a Full Stack
- External Events
 - $\hat{C}$
 - Socket Event
- Logical Errors
 - Pop from an Empty Stack
 - Resource Errors – like Memory Read/Write
- Run time Errors
 - Arithmetic Overflow / Underflow
 - Out of Range
- Undefined Operation
 - Division by Zero



Exception Handling?

Module M36

Partha Pratim Das

Weekly Recap

Objective & Outline

Exception Fundamentals

Types of Exceptions

Exception Stages

Error Handling in C

C Language Features

RV & Params

Local goto

C Standard Library Support

Global Variables

Abnormal Termination

Conditional Termination

Non-Local goto

Signals

Shortcomings

Module Summary

- Exception Handling is a mechanism that separates the detection and handling of circumstantial **Exceptional Flow** from **Normal Flow**
- Current state saved in a pre-defined location
- Execution switched to a pre-defined handler

Exceptions are C++'s means of separating error reporting from error handling

– Bjarne Stroustrup



Types of Exceptions

Module M36

Partha Pratim Das

Weekly Recap

Objective & Outline

Exception Fundamentals

Types of Exceptions

Exception Stages

Error Handling in C

C Language Features

RV & Params

Local goto

C Standard Library Support

Global Variables

Abnormal Termination

Conditional Termination

Non-Local goto

Signals

Shortcomings

Module Summary

- **Asynchronous Exceptions:**

- Exceptions that come Unexpectedly
- Example – an Interrupt in a Program
- Takes control away from the Executing Thread context to a context that is different from that which caused the exception

- **Synchronous Exceptions:**

- Planned Exceptions
- Handled in an organized manner
- The most common type of Synchronous Exception is implemented as a **throw**



Exception Stages

Module M36

Partha Pratim Das

Weekly Recap

Objective & Outline

Exception Fundamentals

Types of Exceptions

Exception Stages

Error Handling in C

C Language Features

RV & Params

Local goto

C Standard Library Support

Global Variables

Abnormal Termination

Conditional Termination

Non-Local goto

Signals

Shortcomings

Module Summary

[1] Error Incidence

- **Synchronous** (S/W) Logical Error
- **Asynchronous** (H/W) Interrupt (S/W Interrupt)

[2] Create Object & Raise Exception

- An Exception Object can be of any Complete Type - an `int` to a full blown `C++ class object`

[3] Detect Exception

- **Polling** – Software Tests
- **Notification** – Control (Stack) Adjustments

[4] Handle Exception

- **Ignore**: hope someone else handles it, that is, Do Not Catch
- **Act**: but allow others to handle it afterwards, that is, Catch, Handle and Re-Throw
- **Own**: take complete ownership, that is, Catch and Handle

[5] Recover from Exception

- **Continue Execution**: If handled inside the program
- **Abort Execution**: If handled outside the program



Exception Stages

Module M36

Partha Pratim
Das

Weekly Recap

Objective &
Outline

Exception
Fundamentals

Types of Exceptions

Exception Stages

Error Handling in
C

C Language Features

RV & Params

Local goto

C Standard Library
Support

Global Variables

Abnormal
Termination

Conditional
Termination

Non-Local goto

Signals

Shortcomings

Module Summary

```
int f() {  
    int error;  
    /* ... */  
    if (error) /* Stage 1: error occurred */  
        return -1; /* Stage 2: generate exception object */  
    /* ... */  
}  
  
int main(void) {  
    if (f() != 0) /* Stage 3: detect exception */  
    {  
        /* Stage 4: handle exception */  
    }  
    /* Stage 5: recover */  
}
```



Error Handling in C

Module M36

Partha Pratim
Das

Weekly Recap

Objective &
Outline

Exception
Fundamentals

Types of Exceptions
Exception Stages

**Error Handling in
C**

C Language Features

RV & Params

Local goto

C Standard Library
Support

Global Variables

Abnormal
Termination

Conditional
Termination

Non-Local goto

Signals

Shortcomings

Module Summary

Error Handling in C



Support for Error Handling in C

Module M36

Partha Pratim Das

Weekly Recap

Objective & Outline

Exception Fundamentals

Types of Exceptions

Exception Stages

Error Handling in C

C Language Features

RV & Params

Local goto

C Standard Library Support

Global Variables

Abnormal Termination

Conditional Termination

Non-Local goto

Signals

Shortcomings

Module Summary

- Support for Error Handling in C

- C language does not provide any specific feature for error handling. Consequently, developers are forced to use normal programming features in a disciplined way to handle errors. This has led to industry practices that the developers should abide by
- C Standard Library provides a collection of headers that can be used for handling errors in different contexts. None of them is complete in itself, but together they kind of cover most situations. This again has led to industry practices that the developers should follow

- Language Features

- Return Value & Parameters
- Local **goto**

- Standard Library Support

- Global Variables (**<errno.h>**)
- Abnormal Termination (**<stdlib.h>**)
- Conditional Termination (**<assert.h>**)
- Non-Local **goto** (**<setjmp.h>**)
- Signals (**<signal.h>**)



Return Value & Parameters

Module M36

Partha Pratim Das

Weekly Recap

Objective & Outline

Exception Fundamentals

Types of Exceptions

Exception Stages

Error Handling in C

C Language Features

RV & Params

Local goto

C Standard Library Support

Global Variables

Abnormal Termination

Conditional Termination

Non-Local goto

Signals

Shortcomings

Module Summary

- Function Return Value Mechanism
 - Created by the Callee as Temporary Objects
 - Passed onto the Caller
 - Caller checks for Error Conditions
 - Return Values can be ignored and lost
 - Return Values are temporary
- Function (output) Parameter Mechanism
 - Outbound Parameters
 - Bound to arguments
 - Offer multiple logical Return Values



Example: Return Value & Parameters

Module M36

Partha Pratim
Das

Weekly Recap

Objective &
Outline

Exception
Fundamentals

Types of Exceptions
Exception Stages

Error Handling in
C

C Language Features

RV & Params

Local goto

C Standard Library
Support

Global Variables

Abnormal
Termination

Conditional
Termination

Non-Local goto

Signals

Shortcomings

Module Summary

```
int Push(int i) {  
    if (top_ == size-1) // Incidence  
        return 0; // Raise  
    else  
        stack_[++top_] = i;  
  
    return 1;  
}  
  
int main() {  
    int x;  
    // ...  
    if (!Push(x)) { // Detect  
        // Handling  
    }  
    // Recovery  
}
```



Local goto

Module M36

Partha Pratim Das

Weekly Recap

Objective & Outline

Exception Fundamentals

Types of Exceptions
Exception Stages

Error Handling in C

C Language Features

RV & Params

Local goto

C Standard Library Support

Global Variables

Abnormal Termination

Conditional Termination

Non-Local goto

Signals

Shortcomings

Module Summary

- Local goto Mechanism
 - (At Source) *Escapes*: Gets Control out of a Deep Nested Loop
 - (At Destination) *Refactors*: Actions from Multiple Points of Error Inception
- A group of C Features
 - `goto` Label;
 - `break continue`;
 - `default switch case`



Example: Local goto

Module M36

Partha Pratim
Das

Weekly Recap

Objective &
Outline

Exception
Fundamentals

Types of Exceptions

Exception Stages

Error Handling in
C

C Language Features

RV & Params

Local goto

C Standard Library
Support

Global Variables

Abnormal
Termination

Conditional
Termination

Non-Local goto

Signals

Shortcomings

Module Summary

```
_PHNDLR _cdecl signal(int signum, _PHNDLR sigact)
{ // Lifted from VC98\CRT\SRC\WINSIG.C
...    /* Check for sigact support */
        if ( (sigact == ...) ) goto sigreterror;

        /* Not exceptions in the host OS. */
        if ( (signum == ...) ) { ... goto sigreterror; }
        else { ... goto sigretok; }

        /* Exceptions in the host OS. */
        if ( (signum ...) ) goto sigreterror;

...
sigretok:
    return(oldsigact);

sigreterror:
    errno = EINVAL;
    return(SIG_ERR);
}
```



Example: Local goto

Module M36

Partha Pratim
Das

Weekly Recap

Objective &
Outline

Exception
Fundamentals

Types of Exceptions
Exception Stages

Error Handling in
C

C Language Features
RV & Params

Local goto

C Standard Library
Support

Global Variables

Abnormal
Termination

Conditional
Termination

Non-Local goto

Signals

Shortcomings

Module Summary

```
_PHNDLR __cdecl signal(int signum, _PHNDLR sigact)
{ // Lifted from VC98\CRT\SRC\WINSIG.C
...    /* Check for sigact support */
        if ( (sigact == ...) ) goto sigreterror;

        /* Not exceptions in the host OS. */
        if ( (signum == ...) ) { ... goto sigreterror; }
        else { ... goto sigretok; }

        /* Exceptions in the host OS. */
        if ( (signum ...) ) goto sigreterror;
...
sigretok:
    return(oldsigact);

sigreterror:
    errno = EINVAL;
    return(SIG_ERR);
}
```



Example: Local goto

Module M36

Partha Pratim Das

Weekly Recap

Objective & Outline

Exception Fundamentals

Types of Exceptions
Exception Stages

Error Handling in C

C Language Features

RV & Params

Local goto

C Standard Library Support

Global Variables

Abnormal Termination

Conditional Termination

Non-Local goto

Signals

Shortcomings

Module Summary

```
_PHNDLR __cdecl signal(int signum, _PHNDLR sigact)
{ // Lifted from VC98\CRT\SRC\WINSIG.C
...    /* Check for sigact support */
        if ( (sigact == ...) ) goto sigreterror;

        /* Not exceptions in the host OS. */
        if ( (signum == ...) ) { ... goto sigreterror; }
        else { ... goto sigretok; }

        /* Exceptions in the host OS. */
        if ( (signum ...) ) goto sigreterror;
...
sigretok:
    return(oldsigact);

sigreterror:
    errno = EINVAL;
    return(SIG_ERR);
}
```



Example: Local goto

Module M36

Partha Pratim Das

Weekly Recap

Objective & Outline

Exception Fundamentals

Types of Exceptions
Exception Stages

Error Handling in C

C Language Features

RV & Params

Local goto

C Standard Library Support

Global Variables

Abnormal Termination

Conditional Termination

Non-Local goto

Signals

Shortcomings

Module Summary

```
_PHNDLR __cdecl signal(int signum, _PHNDLR sigact)
{ // Lifted from VC98\CRT\SRC\WINSIG.C
...    /* Check for sigact support */
        if ( (sigact == ...) ) goto sigreterror;

        /* Not exceptions in the host OS. */
        if ( (signum == ...) ) { ... goto sigreterror; }
        else { ... goto sigretok; }

        /* Exceptions in the host OS. */
        if ( (signum ...) ) goto sigreterror;
...
sigretok:
    return(oldsigact);

sigreterror:
    errno = EINVAL;
    return(SIG_ERR);
}
```



Global Variables

Module M36

Partha Pratim Das

Weekly Recap

Objective & Outline

Exception Fundamentals

Types of Exceptions

Exception Stages

Error Handling in C

C Language Features

RV & Params

Local goto

C Standard Library Support

Global Variables

Abnormal Termination

Conditional Termination

Non-Local goto

Signals

Shortcomings

Module Summary

- GV Mechanism
 - Use a designated Global Error Variable
 - Set it on Error
 - Poll / Check it for Detection
- Standard Library GV Mechanism
 - `<errno.h>/<cerrno>`



Example: Global Variables

Module M36

Partha Pratim
Das

Weekly Recap

Objective &
Outline

Exception
Fundamentals

Types of Exceptions
Exception Stages

Error Handling in
C

C Language Features

RV & Params

Local goto

C Standard Library
Support

Global Variables

Abnormal
Termination

Conditional
Termination

Non-Local goto

Signals

Shortcomings

Module Summary

```
#include <errno.h>
#include <math.h>
#include <stdio.h>
```

```
int main() {
    double x, y, result;
    /*... somehow set 'x' and 'y' ...*/
    errno = 0;

    result = pow(x, y);

    if (errno == EDOM)
        printf("Domain error on x/y pair \n");
    else
        if (errno == ERANGE)
            printf("range error in result \n");
        else
            printf("x to the y = %d \n", (int) result);
}
```



Abnormal Termination

Module M36

Partha Pratim Das

Weekly Recap

Objective & Outline

Exception Fundamentals

Types of Exceptions
Exception Stages

Error Handling in C

C Language Features

RV & Params

Local goto

C Standard Library Support

Global Variables

Abnormal Termination

Conditional Termination

Non-Local goto

Signals

Shortcomings

Module Summary

- Program Halting Functions provided by
 - `<stdlib.h>/<cstdlib>`
- `abort()`
 - Catastrophic Program Failure
- `exit()`
 - Code Clean up via `atexit()` Registrations
- `atexit()`
 - Handlers called in reverse order of their Registrations



Example: Abnormal Termination

Module M36

Partha Pratim
Das

Weekly Recap

Objective &
Outline

Exception
Fundamentals

Types of Exceptions
Exception Stages

Error Handling in
C

C Language Features
RV & Params

Local goto

C Standard Library
Support

Global Variables

**Abnormal
Termination**

Conditional
Termination

Non-Local goto

Signals

Shortcomings

Module Summary

```
#include <stdio.h>
#include <stdlib.h>

static void atexit_handler_1(void) {
    printf("within 'atexit_handler_1' \n");
}

static void atexit_handler_2(void) {
    printf("within 'atexit_handler_2' \n");
}

int main() {
    atexit(atexit_handler_1);
    atexit(atexit_handler_2);
    exit(EXIT_SUCCESS);

    printf("This line should never appear \n");

    return 0;
}

within 'atexit_handler_2'
within 'atexit_handler_1'
```




Conditional Termination

Module M36

Partha Pratim Das

Weekly Recap

Objective & Outline

Exception Fundamentals

Types of Exceptions
Exception Stages

Error Handling in C

C Language Features

RV & Params

Local goto

C Standard Library Support

Global Variables

Abnormal Termination

Conditional Termination

Non-Local goto

Signals

Shortcomings

Module Summary

- Diagnostic ASSERT macro defined in
 - `<assert.h>/<cassert>`
- Assertions valid when `NDEBUG` macro is not defined (debug build is done)
- Assert calls internal function, reports the source file details and then Terminates



Example: Conditional Termination

Module M36

Partha Pratim
Das

Weekly Recap

Objective &
Outline

Exception
Fundamentals

Types of Exceptions
Exception Stages

Error Handling in
C

C Language Features

RV & Params
Local goto

C Standard Library
Support

Global Variables

Abnormal
Termination

Conditional
Termination

Non-Local goto

Signals

Shortcomings

Module Summary

```
/* Debug version */
#define NDEBUG
#include <assert.h>
#include <stdlib.h>
#include <stdio.h>

/* When run - Asserts */
int main() { int i = 0;
    assert(++i == 0); // Assert 0 here

    printf(" i is %d \n", i);

    return 0;
}

void _assert(int test, char const * test_image, char const * file, int line) {
    if (!test) { printf("assertion failed: %s , file %s , line %d\n", test_image, file, line);
        abort();
    }
}
```

Assertion failed: ++i == 0, // On MSVC++
file d:\ppd\my courses...\codes\msvc\programming in modern c++\exception in c\assertion.c,
line 8

a.out: main.c:17: main: Assertion '++i == 0' failed. // On onlinegdb

Programming in Modern C++

Partha Pratim Das

M36.26



Example: Conditional Termination (On MSVC++)

Module M36

Partha Pratim Das

Weekly Recap

Objective & Outline

Exception Fundamentals

Types of Exceptions

Exception Stages

Error Handling in C

C Language Features

RV & Params

Local goto

C Standard Library Support

Global Variables

Abnormal

Termination

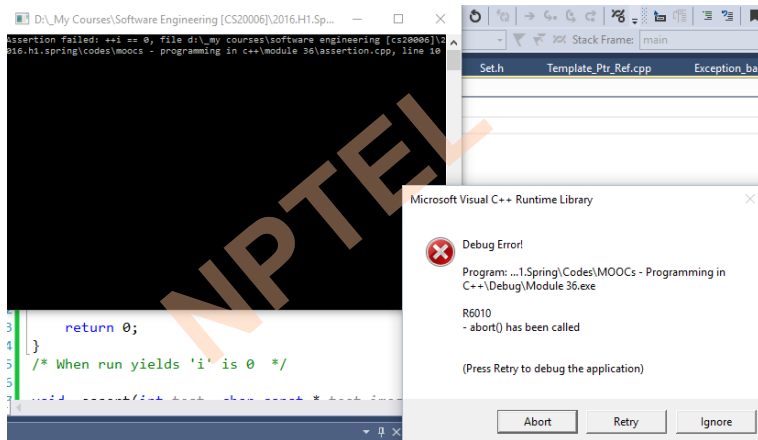
Conditional Termination

Non-Local goto

Signals

Shortcomings

Module Summary





Example: Conditional Termination

Module M36

Partha Pratim
Das

Weekly Recap

Objective &
Outline

Exception
Fundamentals

Types of Exceptions

Exception Stages

Error Handling in
C

C Language Features

RV & Params

Local goto

C Standard Library
Support

Global Variables

Abnormal
Termination

Conditional
Termination

Non-Local goto

Signals

Shortcomings

Module Summary

```
/* Release version */
#define NDEBUG
#include <assert.h>
#include <stdlib.h>
#include <stdio.h>

/* When run yields 'i' is 0 */
int main() {
    int i = 0;
    assert(++i == 0); // Assert 0 here

    printf(" i is %d \n", i);

    return 0;
}

void _assert(int test, char const * test_image, char const * file, int line) {
    if (!test) {
        printf("assertion failed: %s , file %s , line %d\n", test_image, file, line);
        abort();
    }
}

i is 0
```



Non-Local goto

Module M36

Partha Pratim Das

Weekly Recap

Objective & Outline

Exception Fundamentals

Types of Exceptions

Exception Stages

Error Handling in C

C Language Features

RV & Params

Local goto

C Standard Library Support

Global Variables

Abnormal Termination

Conditional Termination

Non-Local goto

Signals

Shortcomings

Module Summary

- `setjmp()` and `longjmp()` functions provided in `<setjmp.h>` Header along with collateral type `jmp_buf`
- `setjmp(jmp_buf)`
 - Sets the Jump point filling up the `jmp_buf` object with the current program context
- `longjmp(jmp_buf, int)`
 - Effects a Jump to the context of the `jmp_buf` object
 - Control return to `setjmp` call last called on `jmp_buf`



Example: Non-Local goto: The Dynamics

Module M36

Partha Pratim Das

Weekly Recap

Objective & Outline

Exception Fundamentals

Types of Exceptions

Exception Stages

Error Handling in C

C Language Features

RV & Params

Local goto

C Standard Library Support

Global Variables

Abnormal Termination

Conditional Termination

Non-Local goto

Signals

Shortcomings

Module Summary

Caller

```
#include <stdio.h>
#include <stdbool.h>
#include <setjmp.h>

int main() {
    if (setjmp(jbuf) == 0) {
        printf("g() called\n");
        g();
        printf("g() returned\n");
    }
    else
        printf("g() failed\n");
    return 0;
}
```

Callee

```
jmp_buf jbuf;

void g() {
    bool error = false;
    printf("g() started\n");
    if (error)
        longjmp(jbuf, 1);
    printf("g() ended\n");
    return;
}
```



Example: Non-Local goto: The Dynamics

Module M36

Partha Pratim Das

Weekly Recap

Objective & Outline

Exception Fundamentals

Types of Exceptions

Exception Stages

Error Handling in C

C Language Features

RV & Params

Local goto

C Standard Library Support

Global Variables

Abnormal Termination

Conditional Termination

Non-Local goto

Signals

Shortcomings

Module Summary

Caller	Callee
<pre>int main() { if (setjmp(jbuf) == 0) { printf("g() called\n"); g(); printf("g() returned\n"); } else printf("g() failed\n"); return 0; }</pre>	<pre>jmp_buf jbuf; void g() { bool error = false; printf("g() started\n"); if (error) longjmp(jbuf, 1); printf("g() ended\n"); return; }</pre>
(1) <code>g()</code> called	(2) <code>g()</code> successfully returned

`g() called`
`g() started`
`g() ended`
`g() returned`



Example: Non-Local goto: The Dynamics

Module M36

Partha Pratim Das

Weekly Recap

Objective & Outline

Exception Fundamentals

Types of Exceptions

Exception Stages

Error Handling in C

C Language Features

RV & Params

Local goto

C Standard Library Support

Global Variables

Abnormal Termination

Conditional Termination

Non-Local goto

Signals

Shortcomings

Module Summary

Caller	Callee
<pre>int main() { if (setjmp(jbuf) == 0) { printf("g() called\n"); g(); printf("g() returned\n"); } else printf("g() failed\n"); return 0; }</pre>	<pre>jmp_buf jbuf; void g() { bool error = true; printf("g() started\n"); if (error) longjmp(jbuf, 1); printf("g() ended\n"); return; }</pre>
(1) <code>g()</code> called (3) <code>setjmp</code> takes to handler <code>g() called</code> <code>g() started</code> <code>g() failed</code>	(2) <code>longjmp</code> executed



Example: Non-Local goto

Module M36

Partha Pratim
Das

Weekly Recap

Objective &
Outline

Exception
Fundamentals

Types of Exceptions

Exception Stages

Error Handling in
C

C Language Features

RV & Params

Local goto

C Standard Library
Support

Global Variables

Abnormal
Termination

Conditional
Termination

Non-Local goto

Signals

Shortcomings

Module Summary

```
#include <setjmp.h>
#include <stdio.h>

jmp_buf j;

void raise_exception() {
    printf("Exception raised. \n");
    longjmp(j, 1); /* Jump to exception handler */
    printf("This line should never appear \n");
}

int main() {
    if (setjmp(j) == 0) {
        printf("'setjmp' is initializing j. \n");
        raise_exception();
        printf("This line should never appear \n");
    }
    else
        printf("'setjmp' was just jumped into. \n");
    /* The exception handler code here */

    return 0 ;
}
```

```
'setjmp' is initializing j.
Exception raised.
'setjmp' was just jumped into.
```

Programming in Modern C++

Partha Pratim Das

M36.33



Signals

Module M36

Partha Pratim Das

Weekly Recap

Objective & Outline

Exception Fundamentals

Types of Exceptions

Exception Stages

Error Handling in C

C Language Features

RV & Params

Local goto

C Standard Library Support

Global Variables

Abnormal Termination

Conditional Termination

Non-Local goto

Signals

Shortcomings

Module Summary

- Header `<signal.h>`
- `raise()`
 - Sends a signal to the executing program
- `signal()`
 - Registers interrupt signal handler
 - Returns the previous handler associated with the given signal
- Converts h/w interrupts to s/w interrupts



Example: Signals

Module M36

Partha Pratim Das

Weekly Recap

Objective & Outline

Exception Fundamentals

Types of Exceptions

Exception Stages

Error Handling in C

C Language Features

RV & Params

Local goto

C Standard Library Support

Global Variables

Abnormal Termination

Conditional Termination

Non-Local goto

Signals

Shortcomings

Module Summary

```
// Use signal to attach a signal
// handler to the abort routine
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>

void SignalHandler(int signal) {
    printf("Application aborting...\n");
}

int main() {
    typedef void (*SignalHandlerPointer)(int);

    SignalHandlerPointer previousHandler;

    previousHandler = signal(SIGABRT, SignalHandler);

    abort();

    return 0;
}
```

Application aborting...



Shortcomings

Module M36

Partha Pratim Das

Weekly Recap

Objective & Outline

Exception Fundamentals

Types of Exceptions
Exception Stages

Error Handling in C

C Language Features

RV & Params

Local goto

C Standard Library Support

Global Variables

Abnormal Termination

Conditional Termination

Non-Local goto

Signals

Shortcomings

Module Summary

- **Destructor-ignorant:**
 - Cannot release Local Objects i.e. Resources Leak
- **Obtrusive:**
 - Interrogating RV or GV results in Code Clutter
- **Inflexible:**
 - Spoils Normal Function Semantics
- **Non-native:**
 - Require Library Support outside Core Language



Module Summary

Module M36

Partha Pratim
Das

Weekly Recap

Objective &
Outline

Exception
Fundamentals

Types of Exceptions
Exception Stages

Error Handling in
C

C Language Features

RV & Params

Local goto

C Standard Library
Support

Global Variables

Abnormal
Termination

Conditional
Termination

Non-Local goto

Signals

Shortcomings

Module Summary

- Introduced the concept of exceptions
- Discussed error handling in C
- Illustrated various language features and library support in C for handling errors
- Demonstrated with examples



Module M37

Partha Pratim
Das

Objectives &
Outlines

Exceptions in
C++

try-throw-catch

Exception Scope
(try)

Exception Arguments
(catch)

Exception Matching

Exception Raise
(throw)

Advantages

std::exception

Module Summary

Programming in Modern C++

Module M37: Exceptions (Error handling in C++): Part 2

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Module Recap

Module M37

Partha Pratim
Das

Objectives & Outlines

Exceptions in
C++

try-throw-catch

Exception Scope
(try)

Exception Arguments
(catch)

Exception Matching

Exception Raise
(throw)

Advantages

std::exception

Module Summary

- Introduced the concept of exceptions
- Discussed error handling in C
- Illustrated various language features and library support in C for handling errors
- Demonstrated with examples



Module Objectives

Module M37

Partha Pratim
Das

Objectives & Outlines

Exceptions in
C++

try-throw-catch

Exception Scope
(try)

Exception Arguments
(catch)

Exception Matching

Exception Raise
(throw)

Advantages

std::exception

Module Summary

- Understand the Error handling in C++

NPTEL



Module Outline

Module M37

Partha Pratim
Das

Objectives & Outlines

Exceptions in
C++

try-throw-catch

Exception Scope
(try)

Exception Arguments
(catch)

Exception Matching

Exception Raise
(throw)

Advantages

std::exception

Module Summary

- 1 Exceptions in C++
 - try-throw-catch
 - Exception Scope (try)
 - Exception Arguments (catch)
 - Exception Matching
 - Exception Raise (throw)
 - Advantages
 - std::exception

- 2 Module Summary



Exceptions in C++

Module M37

Partha Pratim
Das

Objectives &
Outlines

Exceptions in
C++

try-throw-catch

Exception Scope
(try)

Exception Arguments
(catch)

Exception Matching

Exception Raise
(throw)

Advantages

std::exception

Module Summary

Exceptions in C++



Expectations

Module M37

Partha Pratim
Das

Objectives &
Outlines

Exceptions in
C++

try-throw-catch

Exception Scope
(try)

Exception Arguments
(catch)

Exception Matching

Exception Raise
(throw)

Advantages

std::exception

Module Summary

- Separate *Error-Handling code* from *Normal code*
- *Language Mechanism* rather than of the Library
- Compiler for *Tracking Automatic Variables*
- Schemes for *Destruction of Dynamic Memory*
- *Less Overhead* for the Designer
- *Exception Propagation* from the deepest of levels
- *Various Exceptions* handled by a single Handler



Error Handling Dynamics: C and C++

Module M37

Partha Pratim Das

Objectives & Outlines

Exceptions in C++

try-throw-catch

Exception Scope (try)

Exception Arguments (catch)

Exception Matching

Exception Raise (throw)

Advantages

std::exception

Module Summary

Header

Caller

Callee

C Scenario

```
#include <stdio.h>
#include <stdbool.h>
#include <setjmp.h>
```

```
int main() {
    if (setjmp(jbuf) == 0) {
        printf("g() called\n");
        g();
        printf("g() returned\n");
    }
    else printf("g() failed\n"); // On longjmp
    return 0;
}
```

```
jmp_buf jbuf;
void g() {
    bool error = false;
    printf("g() started\n");
    if (error)
        longjmp(jbuf, 1);
    printf("g() ended\n");
    return;
}
```

C++ Scenario

```
#include <iostream>
#include <exception>
using namespace std;
```

```
int main() {
    try {
        cout << "g() called\n";
        g();
        cout << "g() returned\n";
    }
    catch (Excp&) { cout << "g() failed\n"; }
    return 0;
}
```

```
class Excp: public exception {};
void g() {
    bool error = false;
    cout << "g() started\n";
    if (error)
        throw Excp();
    cout << "g() ended\n";
    return;
}
```



try-throw-catch

Module M37

Partha Pratim Das

Objectives & Outlines

Exceptions in C++

try-throw-catch

Exception Scope (try)

Exception Arguments (catch)

Exception Matching

Exception Raise (throw)

Advantages

std::exception

Module Summary

Caller

```
int main() {  
    try {  
        cout << "g() called\n";  
        g();  
        cout << "g() returned\n";  
    }  
    catch (Excp&) { cout << "g() failed\n"; }  
    return 0;  
}
```

(1) g() called

g() called
g() started
g() ended
g() returned

Callee

```
class Excp: public exception {};  
void g() {  
    bool error = false;  
    cout << "g() started\n";  
    if (error)  
        throw Excp();  
    cout << "g() ended\n";  
    return;  
}
```

(2) g() successfully returned



try-throw-catch

Module M37

Partha Pratim Das

Objectives & Outlines

Exceptions in C++

try-throw-catch

Exception Scope (try)

Exception Arguments (catch)

Exception Matching

Exception Raise (throw)

Advantages

std::exception

Module Summary

Caller	Callee
<pre>int main() { try { cout << "g() called\n"; g(); cout << "g() returned\n"; } catch (Excp&) { cout << "g() failed\n"; } return 0; }</pre>	<pre>class Excp: public exception {}; class A {}; void g() { A a; bool error = true; cout << "g() started\n"; if (error) throw Excp(); cout << "g() ended\n"; return; }</pre>
(1) g() called (5) Exception caught by catch clause (6) Normal flow continues	(2) Exception raised (3) Stack frame of g() unwinds and destructor of a called (4) Remaining execution of g() and cout skipped

g() called
g() started
g() failed



Exception Flow

Module M37

Partha Pratim
Das

Objectives &
Outlines

Exceptions in
C++

try-throw-catch

Exception Scope
(try)

Exception Arguments
(catch)

Exception Matching

Exception Raise
(throw)

Advantages

std::exception

Module Summary

```
#include <iostream>
#include <exception>
using namespace std;
class MyException: public exception { };
class MyClass { public: ~MyClass() { } };
void h() { MyClass h_a;
    //throw 1;           // Line 1
    //throw 2.5;         // Line 2
    //throw MyException(); // Line 3
    //throw exception();  // Line 4
    //throw MyClass();    // Line 5
} // Stack unwind, h_a.~MyClass() called
// Passes on all exceptions
void g() { MyClass g_a;
    try { h();
        bool okay = true; // Not executed
    }
    // Catches exception from Line 1
    catch (int) { cout << "int\n"; }
    // Catches exception from Line 2
    catch (double) { cout << "double\n"; }
    // Catches exception from Line 3-5 & passes on
    catch (...) { throw; }
} // Stack unwind, g_a.~MyClass() called
```

```
void f() { MyClass f_a;
    try { g();
        bool okay = true; // Not executed
    }
    // Catches exception from Line 3
    catch (MyException) { cout << "MyException\n"; }
    // Catches exception from Line 4
    catch (exception) { cout << "exception\n"; }
    // Catches exception from Line 5 & passes on
    catch (...) { throw; }
} // Stack unwind, f_a.~MyClass() called

int main() {
    try { f();
        bool okay = true; // Not executed
    }
    // Catches exception from Line 5
    catch (...) { cout << "Unknown\n"; }

    cout << "End of main()\n";
}
```



try Block: Exception Scope

- **try** block
 - Consolidate areas that might throw exceptions
- function **try** block
 - Area for detection is the entire function body
- Nested **try** block
 - Semantically equivalent to nested function calls

Function **try**

```
void f()  
    try {  
        throw E();  
    }  
    catch (E& e) {  
    }
```

Nested **try**

```
try {  
    try { throw E(); }  
    catch (E& e) { }  
}  
catch (E& e1) {  
}
```

Note: The usual curly braces for the function scope are not to be put here



catch Block: Exception Arguments

Module M37

Partha Pratim
Das

Objectives &
Outlines

Exceptions in
C++

try-throw-catch

Exception Scope
(try)

Exception Arguments
(catch)

Exception Matching

Exception Raise
(throw)

Advantages

std::exception

Module Summary

- **catch** block
 - Name for the Exception Handler
 - Catching an Exception is like invoking a function
 - Immediately follows the **try** block
 - Unique Formal Parameter for each Handler
 - Can simply be a Type Name to distinguish its Handler from others



try-catch: Exception Matching

Module M37

Partha Pratim Das

Objectives & Outlines

Exceptions in C++

try-throw-catch

Exception Scope (try)

Exception Arguments (catch)

Exception Matching

Exception Raise (throw)

Advantages

std::exception

Module Summary

- **Exact Match**
 - The catch argument type matches the type of the thrown object
 - ▷ *No implicit conversion is allowed*
- **Generalization / Specialization**
 - The catch argument is a public base class of the thrown class object
- **Pointer**
 - Pointer types – convertible by standard conversion



try-catch: Exception Matching

Module M37

Partha Pratim Das

Objectives & Outlines

Exceptions in C++

try-throw-catch

Exception Scope (try)

Exception Arguments (catch)

Exception Matching

Exception Raise (throw)

Advantages

std::exception

Module Summary

- In the *order of appearance* with matching
- If Base Class `catch` block precedes Derived Class `catch` block
 - Compiler issues a warning and continues
 - Unreachable code (derived class handler) ignored
- `catch(...)` block must be the last `catch` block because it catches all exceptions
- If no matching Handler is found in the current scope, the search continues to find a matching handler in a dynamically surrounding `try` block
 - *Stack Unwinds*
- If eventually no handler is found, `terminate()` is called



throw *Expression*: Exception Raise

Module M37

Partha Pratim Das

Objectives &
Outlines

Exceptions in
C++

try-throw-catch

Exception Scope
(try)

Exception Arguments
(catch)

Exception Matching

Exception Raise
(throw)

Advantages

std::exception

Module Summary

- *Expression* is treated the same way as
 - A *function argument* in a call or the *operand of a return* statement
- Exception Context
 - `class Exception { };`
- The *Expression*
 - Generate an Exception object to throw
 - ▷ `throw Exception();`
 - Or, Copies an existing Exception object to throw
 - ▷ `Exception ex;`
 - ▷ `...`
 - ▷ `throw ex; // Exception(ex);`
- *Exception object is created on the Free Store*



throw *Expression*: Restrictions

Module M37

Partha Pratim Das

Objectives & Outlines

Exceptions in C++

try-throw-catch

Exception Scope (try)

Exception Arguments (catch)

Exception Matching

Exception Raise (throw)

Advantages

std::exception

Module Summary

- For a UDT Expression
 - Copy Constructor and Destructor should be supported
- The type of Expression cannot be an incomplete type or a pointer to an incomplete type
 - No incomplete type like void, array of unknown size or of elements of incomplete type, Declared but not Defined struct / union / enum / class Objects or Pointers to such Objects
 - No pointer to an incomplete type, except void*, const void*, volatile void*, const volatile void*



(re)-throw: Throwing Again?

Module M37

Partha Pratim Das

Objectives & Outlines

Exceptions in C++

try-throw-catch

Exception Scope (try)

Exception Arguments (catch)

Exception Matching

Exception Raise (throw)

Advantages

std::exception

Module Summary

- Re-throw
 - `catch` may pass on the exception after handling
 - Re-`throw` is not same as throwing again!

Throws again

```
try { ... }  
catch (Exception& ex) {  
    // Handle and  
    ...  
    // Raise again  
    throw ex;  
    // ex copied  
    // ex destructed  
}
```

Re-throw

```
try { ... }  
catch (Exception& ex) {  
    // Handle and  
    ...  
    // Pass-on  
    throw;  
    // No copy  
    // No Destruction  
}
```



Advantages

Module M37

Partha Pratim
Das

Objectives &
Outlines

Exceptions in
C++

try-throw-catch

Exception Scope
(try)

Exception Arguments
(catch)

Exception Matching

Exception Raise
(throw)

Advantages

std::exception

Module Summary

- **Destructor-savvy:**
 - Stack unwinds; Orderly destruction of Local-objects
- **Unobtrusive:**
 - Exception Handling is implicit and automatic
 - No clutter of error checks
- **Precise:**
 - Exception Object Type designed using semantics
- **Native and Standard:**
 - EH is part of the C++ language
 - EH is available in all standard C++ compilers



Advantages

Module M37

Partha Pratim
Das

Objectives &
Outlines

Exceptions in
C++

try-throw-catch

Exception Scope
(try)

Exception Arguments
(catch)

Exception Matching

Exception Raise
(throw)

Advantages

std::exception

Module Summary

- **Scalable:**

- Each function can have multiple try blocks
- Each try block can have a single Handler or a group of Handlers
- Each Handler can catch a single type, a group of types, or all types

- **Fault-tolerant:**

- Functions can specify the exception types to throw; Handlers can specify the exception types to catch
- Violation behavior of these specifications is predictable and user-configurable



Exceptions in Standard Library: `std::exception`

Module M37

Partha Pratim
Das

Objectives &
Outlines

Exceptions in
C++

try-throw-catch

Exception Scope
(try)

Exception Arguments
(catch)

Exception Matching

Exception Raise
(throw)

Advantages

`std::exception`

Module Summary

All objects thrown by components of the standard library are derived from this class. Therefore, all standard exceptions can be caught by catching this type by reference.

```
class exception {  
public:  
    exception() throw();  
    exception(const exception&) throw();  
    exception& operator=(const exception&) throw();  
    virtual ~exception() throw();  
    virtual const char* what() const throw();  
}
```

Sources: `std::exception` and `std::exception` in C++11, C++14, C++17 & C++20



Exceptions in Standard Library: `std::exception`

Module M37

Partha Pratim
Das

Objectives &
Outlines

Exceptions in
C++

try-throw-catch

Exception Scope
(try)

Exception Arguments
(catch)

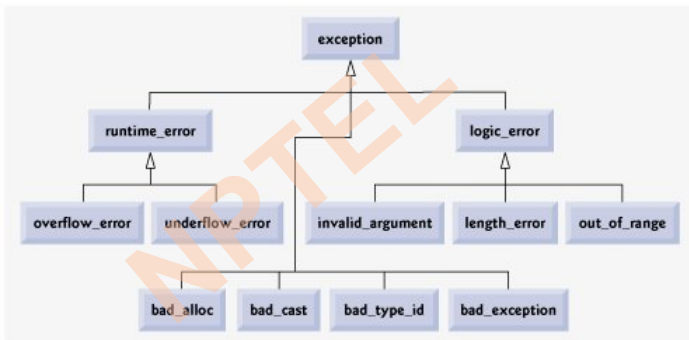
Exception Matching

Exception Raise
(throw)

Advantages

`std::exception`

Module Summary



Sources: [Standard Library Exception Hierarchy](#)



Exceptions in Standard Library: `std::exception`

Module M37

Partha Pratim
Das

Objectives &
Outlines

Exceptions in
C++

try-throw-catch

Exception Scope
(try)

Exception Arguments
(catch)

Exception Matching

Exception Raise
(throw)

Advantages

`std::exception`

Module Summary

- **`logic_error`**: Faulty logic like violating logical preconditions or class invariants (may be preventable)
 - **`invalid_argument`**: An argument value has not been accepted
 - **`domain_error`**: Situations where the inputs are outside of the domain for an operation
 - **`length_error`**: Exceeding implementation defined length limits for some object
 - **`out_of_range`**: Attempt to access elements out of defined range
- **`runtime_error`**: Due to events beyond the scope of the program and can not be easily predicted
 - **`range_error`**: Result cannot be represented by the destination type
 - **`overflow_error`**: Arithmetic overflow errors (Result is too large for the destination type)
 - **`underflow_error`**: Arithmetic underflow errors (Result is a subnormal floating-point value)
- **`bad_typeid`**: Exception thrown on typeid of null pointer
- **`bad_cast`**: Exception thrown on failure to dynamic cast
- **`bad_alloc`**: Exception thrown on failure allocating memory
- **`bad_exception`**: Exception thrown by unexpected handler

Sources: `std::exception` and `std::exception` in C++11, C++14, C++17 & C++20



Exceptions in Standard Library: `std::exception`: C++98, C++11, C++14, C++17 & C++20

Module M37

Partha Pratim Das

Objectives &
Outlines

Exceptions in
C++

try-throw-catch

Exception Scope
(try)

Exception Arguments
(catch)

Exception Matching

Exception Raise
(throw)

Advantages

`std::exception`

Module Summary

- `logic_error`
 - `invalid_argument`
 - `domain_error`
 - `length_error`
 - `out_of_range`
 - `future_error` (C++11)
- `bad_optional_access` (C++17)
- `runtime_error`
 - `range_error`
 - `overflow_error`
 - `underflow_error`
 - `regex_error` (C++11)
 - `system_error` (C++11)
 - ▷ `ios_base::failure` (C++11)
 - ▷ `filesystem::filesystem_error` (C++17)
 - `txtion` (TM TS)
 - `nonexistent_local_time` (C++20)
 - `ambiguous_local_time` (C++20)
 - `format_error` (C++20)
- `bad_typeid`
- `bad_cast`
 - `bad_any_cast` (C++17)
- `bad_weak_ptr` (C++11)
- `bad_function_call` (C++11)
- `bad_alloc`
 - `bad_array_new_length` (C++11)
- `bad_exception`
- `ios_base::failure` (until C++11)
- `bad_variant_access` (C++17)



Module Summary

Module M37

Partha Pratim
Das

Objectives &
Outlines

Exceptions in
C++

`try-throw-catch`

Exception Scope
(`try`)

Exception Arguments
(`catch`)

Exception Matching

Exception Raise
(`throw`)

Advantages

`std::exception`

Module Summary

- Discussed exception (error) handling in C++
- Illustrated `try-throw-catch` feature in C++ for handling errors
- Demonstrated with examples



Module M38

Partha Pratim
Das

Objectives &
Outlines

What is a
Template?

Function
Template

Definition

Instantiation

Template Argument
Deduction

Example

typename

Module Summary

Programming in Modern C++

Module M38: Template (Function Template): Part 1

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Module Recap

Module M38

Partha Pratim
Das

Objectives & Outlines

What is a
Template?

Function
Template

Definition

Instantiation

Template Argument

Deduction

Example

typename

Module Summary

- Discussed exception (error) handling in C++
- Illustrated `try-throw-catch` feature in C++ for handling errors
- Demonstrated with examples



Module Objectives

Module M38

Partha Pratim
Das

Objectives & Outlines

What is a
Template?

Function
Template

Definition

Instantiation

Template Argument

Deduction

Example

typename

Module Summary

- Understand Templates in C++
- Understand Function Templates

NPTEL



Module Outline

Module M38

Partha Pratim
Das

Objectives & Outlines

What is a
Template?

Function
Template

Definition

Instantiation

Template Argument
Deduction

Example

typename

Module Summary

1 What is a Template?

2 Function Template

- Definition
- Instantiation
- Template Argument Deduction
- Example

3 typename

4 Module Summary



What is a Template?

Module M38

Partha Pratim
Das

Objectives &
Outlines

What is a
Template?

Function
Template

Definition

Instantiation

Template Argument

Deduction

Example

typename

Module Summary

What is a Template?



What is a Template?

Module M38

Partha Pratim
Das

Objectives &
Outlines

What is a
Template?

Function
Template

Definition

Instantiation

Template Argument

Deduction

Example

typename

Module Summary

- Templates are specifications of a *collection of functions or classes which are parameterized by types*
- Examples:
 - Function search, min etc.
 - ▷ The basic algorithms in these functions are the same independent of types
 - ▷ Yet, we need to write different versions of these functions for strong type checking in C++
 - Classes list, queue etc.
 - ▷ The data members and the methods are almost the same for list of numbers, list of objects
 - ▷ Yet, we need to define different classes



Function Template

Module M38

Partha Pratim
Das

Objectives &
Outlines

What is a
Template?

Function
Template

Definition

Instantiation

Template Argument
Deduction

Example

typename

Module Summary

Function Template



Function Template: Code reuse in Algorithms

Module M38

Partha Pratim
Das

Objectives &
Outlines

What is a
Template?

Function
Template

Definition

Instantiation

Template Argument

Deduction

Example

typename

Module Summary

- We need to compute the maximum of two values that can be of:

- `int`
- `double`
- `char *` (C-String)
- `Complex` (user-defined class for complex numbers)
- ...

- We can do this with overloaded `Max` functions:

```
int Max(int x, int y);  
double Max(double x, double y);  
char *Max(char *x, char *y);  
Complex Max(Complex x, Complex y);
```

With every new type, we need to add an overloaded function in the library!

- **Issues in `Max` function**

- *Same algorithm* (compare two values using the appropriate operator of the type and return the larger value)
- *Different code versions* of these functions for strong type checking in C++



Max as Overload

Module M38

Partha Pratim Das

Objectives & Outlines

What is a Template?

Function Template

Definition

Instantiation

Template Argument Deduction

Example

typename

Module Summary

```
#include <iostream>
#include <cstring>
#include <cmath>
using namespace std;
// Overloads of Max
int Max(int x, int y) { return x > y ? x : y; }
double Max(double x, double y) { return x > y ? x : y; }
char *Max(char *x, char *y) { return strcmp(x, y) > 0 ? x : y; }

int main() { int a = 3, b = 5, iMax; double c = 2.1, d = 3.7, dMax;
    cout << "Max(" << a << ", " << b << ") = " << Max(a, b) << endl;
    cout << "Max(" << c << ", " << d << ") = " << Max(c, d) << endl;

    char *s1 = new char[6], *s2 = new char[6];
    strcpy(s1, "black"); strcpy(s2, "white");
    cout << "Max(" << s1 << ", " << s2 << ") = " << Max(s1, s2) << endl;
    strcpy(s1, "white"); strcpy(s2, "black");
    cout << "Max(" << s1 << ", " << s2 << ") = " << Max(s1, s2) << endl;
}
```

- Overloaded solutions work
- In some cases (C-string), similar algorithms have exceptions
- With every new type, a new overloaded [Max](#) is needed
- Can we make [Max](#) generic and make a library to work with future types?
- How about macros?



Max as a Macro

Module M38

Partha Pratim Das

Objectives & Outlines

What is a Template?

Function Template

Definition

Instantiation

Template Argument

Deduction

Example

typename

Module Summary

```
#include <iostream>
using namespace std;
```

```
// Max as a macro
```

```
#define Max(x, y) (((x) > (y)) ? x : y)
```

```
int main() {
    int a = 3, b = 5;
    double c = 2.1, d = 3.7;
```

```
    cout << "Max(" << a << ", " << b << ") = " << Max(a, b) << endl; // Output: Max(3, 5) = 5
```

```
    cout << "Max(" << c << ", " << d << ") = " << Max(c, d) << endl; // Output: Max(2.1, 3.7) = 3.7
```

```
    return 0;
```

```
}
```

- **Max**, being a macro, is type oblivious – can be used for **int** as well as **double**, etc.
- Note the parentheses around parameters to protect precedence
- Note the parentheses around the whole expression to protect precedence
- Looks like a function – but does not behave as such



Max as a Macro: Pitfalls

Module M38

Partha Pratim
Das

Objectives &
Outlines

What is a
Template?

Function
Template

Definition

Instantiation

Template Argument
Deduction

Example

typename

Module Summary

```
#include <iostream>
#include <cstring>
using namespace std;

#define Max(x, y) (((x) > (y)) ? x : y)

int main() { int a = 3, b = 5; double c = 2.1, d = 3.7;
    // Side Effects
    cout << "Max(" << a << ", " << b << ") = ";    // Output: Max(3, 5) = 6
    cout << Max(a++, b++) << endl;
    cout << "a = " << a << ", b = " << b << endl; // Output: a = 4, b = 7

    // C-String Comparison
    char *s1 = new char[6], *s2 = new char[6];
    strcpy(s1, "black"); strcpy(s2, "white");
    cout << "Max(" << s1 << ", " << s2 << ") = " << Max(s1, s2) << endl; // Max(black, white) = white

    strcpy(s1, "white"); strcpy(s2, "black");
    cout << "Max(" << s1 << ", " << s2 << ") = " << Max(s1, s2) << endl; // Max(white, black) = black
}
```

- In "Side Effects" – the result is wrong, the larger values gets incremented twice
- In "C-String Comparison" – swapping parameters changes the result – actually compares pointers



Function Template

Module M38

Partha Pratim
Das

Objectives &
Outlines

What is a
Template?

Function
Template

Definition

Instantiation

Template Argument
Deduction

Example

typename

Module Summary

- A **function template**
 - describes how a function should be built
 - supplies the definition of the function using some arbitrary types, (as place holders)
 - ▷ a **parameterized** definition
 - can be considered the definition for a **set of overloaded versions** of a function
 - is identified by the **keyword template**
 - ▷ followed by comma-separated list of **parameter** identifiers (each preceded by **keyword class** or **keyword typename**)
 - ▷ enclosed between **<** and **>** delimiters
 - ▷ followed by the signature the function
 - Note that every template parameter is a **built-in type** or **class** – type parameters



Max as a Function Template

Module M38

Partha Pratim Das

Objectives & Outlines

What is a Template?

Function Template

Definition

Instantiation

Template Argument Deduction

Example

typename

Module Summary

```
#include <iostream>
using namespace std;

template<class T>
T Max(T x, T y) {
    return x > y ? x : y;
}

int main() {
    int a = 3, b = 5, iMax;
    double c = 2.1, d = 3.7, dMax;

    iMax = Max<int>(a, b);
    cout << "Max(" << a << ", " << b << ") = " << iMax << endl; // Output: Max(3, 5) = 5

    dMax = Max<double>(c, d);
    cout << "Max(" << c << ", " << d << ") = " << dMax << endl; // Output: Max(2.1, 3.7) = 3.7
}
```

- **Max**, now, knows the type
- Template type parameter **T** explicitly specified in instantiation of **Max<int>**, **Max<double>**



Max as a Function Template: Pitfall "Side Effects" – Solved

Module M38

Partha Pratim
Das

Objectives &
Outlines

What is a
Template?

Function
Template

Definition

Instantiation

Template Argument
Deduction

Example

typename

Module Summary

```
#include <iostream>
using namespace std;

template<class T>
T Max(T x, T y) {
    return x > y ? x : y;
}

int main() {
    int a = 3, b = 5, iMax;

    // Side Effects
    cout << "Max(" << a << ", " << b << ") = ";
    iMax = Max<int>(a++, b++);
    cout << iMax << endl; // Output: Max(3, 5) = 5

    cout << "a = " << a << ", b = " << b << endl; // Output: a = 4, b = 6
}
```

- **Max** is now a proper function call – no side effect



Max as a Function Template: Pitfall "C-String Comparison" – Solved

Module M38

Partha Pratim Das

Objectives & Outlines

What is a Template?

Function Template

Definition

Instantiation

Template Argument Deduction

Example

typename

Module Summary

```
#include <iostream>
#include <cstring>
using namespace std;

template<class T> T Max(T x, T y) { return x > y ? x : y; }

template<> // Template specialization for 'char*' type
char *Max<char*>(char *x, char *y) { return strcmp(x, y) > 0 ? x : y; }

int main() { char *s1 = new char[6], *s2 = new char[6];
    strcpy(s1, "black"); strcpy(s2, "white");
    cout << "Max(" << s1 << ", " << s2 << ") = " << Max<char*>(s1, s2) << endl;
    // Output: Max(black, white) = white

    strcpy(s1, "white"); strcpy(s2, "black");
    cout << "Max(" << s1 << ", " << s2 << ") = " << Max<char*>(s1, s2) << endl;
    // Output: Max(black, white) = white
}
```

- Generic template code does not work for C-Strings as it compares pointers, not the strings pointed by them
- We provide a specialization to compare pointers using comparison of strings
- Need to specify type explicitly is bothersome



Max as a Function Template: Implicit Instantiation

Module M38

Partha Pratim Das

Objectives & Outlines

What is a Template?

Function Template

Definition

Instantiation

Template Argument Deduction

Example

typename

Module Summary

```
#include <iostream>
using namespace std;

template<class T> T Max(T x, T y) { return x > y ? x : y; }

int main() { int a = 3, b = 5, iMax; double c = 2.1, d = 3.7, dMax;
    iMax = Max(a, b); // Type 'int' inferred from 'a' and 'b' parameters types
    cout << "Max(" << a << ", " << b << ") = " << iMax << endl;
        // Output: Max(3, 5) = 5

    dMax = Max(c, d); // Type 'double' inferred from 'c' and 'd' parameters types
    cout << "Max(" << c << ", " << d << ") = " << dMax << endl;
        // Output: Max(2.1, 3.7) = 3.7
}
```

- Often template type parameter T may be inferred from the type of parameters in the instance
- If the compiler cannot infer or infers wrongly, we use explicit instantiation



Template Argument Deduction: Implicit Instantiation

Module M38

Partha Pratim Das

Objectives & Outlines

What is a Template?

Function Template

Definition

Instantiation

Template Argument Deduction

Example

typename

Module Summary

- Each item in the template parameter list is a template argument
- When a template function is invoked, the values of the template arguments are determined by seeing the types of the function arguments

```
template<class T> T Max(T x, T y);  
template<> char *Max<char *>(char *x, char *y);  
template <class T, int size> T Max(T x[size]);  
  
int a, b; Max(a, b);           // Binds to Max<int>(int, int);  
double c, d; Max(c, d);       // Binds to Max<double>(double, double);  
char *s1, *s2; Max(s1, s2);   // Binds to Max<char*>(char*, char*);  
  
int pval[9]; Max(pval);       // Error!
```

- Three kinds of conversions are allowed
 - L-value transformation (for example, Array-to-pointer conversion)
 - Qualification conversion
 - Conversion to a base class instantiation from a class template
- If the same template parameter are found for more than one function argument, template argument deduction from each function argument must be the same



Max as a Function Template: With User-Defined Class

Module M38

Partha Pratim
Das

Objectives &
Outlines

What is a
Template?

Function
Template

Definition

Instantiation

Template Argument
Deduction

Example

typename

Module Summary

```
#include <iostream>
#include <cmath>
#include <cstring>
using namespace std;

class Complex { double re_; double im_; public:
    Complex(double re = 0.0, double im = 0.0) : re_(re), im_(im) { };
    double norm() const { return sqrt(re_*re_+im_*im_); }
    friend bool operator>(const Complex& c1, const Complex& c2) { return c1.norm() > c2.norm(); }
    friend ostream& operator<<(ostream& os, const Complex& c) {
        os << "(" << c.re_ << ", " << c.im_ << ")"; return os;
    }
};

template<class T> T Max(T x, T y) { return x > y ? x : y; }
template<> char *Max<char *>(char *x, char *y) { return strcmp(x, y) > 0 ? x : y; }

int main() { Complex c1(2.1, 3.2), c2(6.2, 7.2);
    cout << "Max(" << c1 << ", " << c2 << ") = " << Max(c1, c2) << endl;
    // Output: Max((2.1, 3.2), (6.2, 7.2)) = (6.2, 7.2)
}
```

- When `Max` is instantiated with `class Complex`, we need comparison operator for `Complex`
- The code, therefore, will not compile without `bool operator>(const Complex&, const Complex&)`
- Traits of type variable `T` include `bool operator>(T, T)` which the instantiating type must fulfill



Max as a Function Template: Overloads

Module M38

Partha Pratim
Das

Objectives &
Outlines

What is a
Template?

Function
Template

Definition

Instantiation

Template Argument
Deduction

Example

typename

Module Summary

```
#include <iostream>
#include <cstring>
using namespace std;

template<class T> T Max(T x, T y) { return x > y ? x : y; }

template<> char *Max<char *>(char *x, char *y) // Template specialization
{ return strcmp(x, y) > 0 ? x : y; }

template<class T, int size> T Max(T x[size]) { // Overloaded template function
    T t = x[0];
    for (int i = 0; i < size; ++i) { if (x[i] > t) t = x[i]; }

    return t;
}

int main() {
    int arr[] = { 2, 5, 6, 3, 7, 9, 4 };
    cout << "Max(arr) = " << Max<int, 7>(arr) << endl; // Output: Max(arr) = 9
}
```

- Template function can be overloaded
- A template parameter can be non-type ([int](#)) constant



Swap as a Function Template

Module M38

Partha Pratim
Das

Objectives &
Outlines

What is a
Template?

Function
Template

Definition

Instantiation

Template Argument
Deduction

Example

typename

Module Summary

```
#include <iostream>
#include <string>
using namespace std;

template<class T> void Swap(T& one, T& other) { T temp;
    temp = one; one = other; other = temp;
}

int main() { int i = 10, j = 20;
    cout << "i = " << i << ", j = " << j << endl;
    Swap(i, j);
    cout << "i = " << i << ", j = " << j << endl;

    string s1("abc"), s2("def");

    cout << "s1 = " << s1 << ", s2 = " << s2 << endl;
    Swap(s1, s2);
    cout << "s1 = " << s1 << ", s2 = " << s2 << endl;
}
```

- The traits of type variable **T** include
default constructor (**T::T()**) and
copy assignment operator (**T operator=(const T&)**)
- Our template function cannot be called **swap**, as **std** namespace has such a function



typename

Module M38

Partha Pratim
Das

Objectives &
Outlines

What is a
Template?

Function
Template

Definition

Instantiation

Template Argument
Deduction

Example

typename

Module Summary

NPTEL

typename



typename Keyword

Module M38

Partha Pratim
Das

Objectives &
Outlines

What is a
Template?

Function
Template

Definition

Instantiation

Template Argument
Deduction

Example

typename

Module Summary

- Consider:

```
template <class T> f (T x) {  
    T::name * p;  
}
```

- What does it mean?

- `T::name` is a *type* and `p` is a *pointer* to that type
- `T::name` and `p` are *variables* and this is a *multiplication*

- To resolve, we use **keyword** `typename`:

```
template <class T> f (T x) { T::name * p; } // Multiplication
```

```
template <class T> f (T x) { typename T::name * p; } // Type
```

- The keywords `class` and `typename` have almost the same meaning in a template parameter
- `typename` is also used to tell the compiler that an expression is a type expression



Module Summary

Module M38

Partha Pratim
Das

Objectives &
Outlines

What is a
Template?

Function
Template

Definition

Instantiation

Template Argument
Deduction

Example

typename

Module Summary

- Introduced the templates in C++
- Discussed function templates as generic algorithmic solution for code reuse
- Explained templates argument deduction for implicit instantiation
- Illustrated with examples



Module M39

Partha Pratim
Das

Objectives &
Outlines

What is a
Template?

Function
Template

Class Template

Definition

Instantiation

Partial Template

Instantiation &
Default Template
Parameters

Inheritance

Module Summary

Programming in Modern C++

Module M39: Template (Class Template): Part 2

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Module Recap

Module M39

Partha Pratim Das

Objectives & Outlines

What is a Template?

Function Template

Class Template

Definition

Instantiation

Partial Template

Instantiation & Default Template Parameters

Inheritance

Module Summary

- Introduced the templates in C++
- Discussed function templates as generic algorithmic solution for code reuse
- Explained templates argument deduction for implicit instantiation
- Illustrated with examples



Module Objectives

Module M39

Partha Pratim
Das

Objectives & Outlines

What is a
Template?

Function
Template

Class Template

Definition

Instantiation

Partial Template

Instantiation &
Default Template
Parameters

Inheritance

Module Summary

- Understand Templates in C++
- Understand Class Templates

NPTEL



Module Outline

Module M39

Partha Pratim Das

Objectives & Outlines

What is a Template?

Function Template

Class Template

Definition

Instantiation

Partial Template

Instantiation & Default Template Parameters

Inheritance

Module Summary

1 What is a Template?

2 Function Template

3 Class Template

- Definition
- Instantiation
- Partial Template Instantiation & Default Template Parameters
- Inheritance

4 Module Summary



What is a Template?

Module M39

Partha Pratim
Das

Objectives &
Outlines

What is a
Template?

Function
Template

Class Template

Definition

Instantiation

Partial Template

Instantiation &
Default Template
Parameters

Inheritance

Module Summary

What is a Template?



What is a Template?: RECAP (Module 38)

Module M39

Partha Pratim
Das

Objectives &
Outlines

What is a
Template?

Function
Template

Class Template

Definition

Instantiation

Partial Template

Instantiation &

Default Template

Parameters

Inheritance

Module Summary

- Templates are specifications of a collection of functions or classes which are parameterized by types
- Examples:
 - Function search, min etc.
 - ▷ The basic algorithms in these functions are the same independent of types
 - ▷ Yet, we need to write different versions of these functions for strong type checking in C++
 - Classes list, queue etc.
 - ▷ The data members and the methods are almost the same for list of numbers, list of objects
 - ▷ Yet, we need to define different classes



Function Template

Module M39

Partha Pratim
Das

Objectives &
Outlines

What is a
Template?

**Function
Template**

Class Template

Definition

Instantiation

Partial Template

Instantiation &
Default Template
Parameters

Inheritance

Module Summary

Function Template



Function Template: Code reuse in Algorithms: RECAP (Module 38)

Module M39

Partha Pratim Das

Objectives & Outlines

What is a Template?

Function Template

Class Template

Definition

Instantiation

Partial Template

Instantiation & Default Template Parameters

Inheritance

Module Summary

- We need to compute the maximum of two values that can be of:

- `int`
- `double`
- `char *` (C-String)
- `Complex` (user-defined class for complex numbers)
- ...

- We can do this with overloaded `Max` functions:

```
int Max(int x, int y);  
double Max(double x, double y);  
char *Max(char *x, char *y);  
Complex Max(Complex x, Complex y);
```

With every new type, we need to add an overloaded function in the library!

- **Issues in `Max` function**
 - *Same algorithm* (compare two values using the appropriate operator of the type and return the larger value)
 - *Different code versions* of these functions for strong type checking in C++



Class Template

Module M39

Partha Pratim
Das

Objectives &
Outlines

What is a
Template?

Function
Template

Class Template

Definition

Instantiation

Partial Template

Instantiation &
Default Template
Parameters

Inheritance

Module Summary

Class Template



Class Template: Code Reuse in Data Structure

Module M39

Partha Pratim Das

Objectives & Outlines

What is a Template?

Function Template

Class Template

Definition

Instantiation

Partial Template

Instantiation & Default Template

Parameters

Inheritance

Module Summary

- Solution of several problems needs stack (LIFO)
 - Reverse string (char)
 - Convert infix expression to postfix (char)
 - Evaluate postfix expression (int / double / Complex ...)
 - Depth-first traversal (Node *)
 - ...
- Solution of several problems needs queue (FIFO)
 - Task Scheduling (Task *)
 - Process Scheduling (Process *)
 - ...
- Solution of several problems needs list (ordered)
 - Implementing stack, queue (int / char / ...)
 - Implementing object collections (UDT)
 - ...
- Solution of several problems needs ...
- **Issues in Data Structure**
 - Data Structures are **generic - same interface, same algorithms**
 - **C++ implementations are different** due to element type



Stack of char and int

Module M39

Partha Pratim Das

Objectives & Outlines

What is a Template?

Function Template

Class Template

Definition

Instantiation

Partial Template

Instantiation & Default Template

Parameters

Inheritance

Module Summary

```
class Stack {  
    char data_[100];  
    int top_;  
public:  
    Stack() :top_(-1) { }  
    ~Stack() { }  
  
    void push(const char& item) // Has type char  
    { data_[++top_] = item; }  
  
    void pop()  
    { --top_; }  
  
    const char& top() const // Has type char  
    { return data_[top_]; }  
  
    bool empty() const  
    { return top_ == -1; }  
};
```

- Stack of char

- Can we combine these Stack codes using a type variable T?

```
class Stack {  
    int data_[100];  
    int top_;  
public:  
    Stack() :top_(-1) { }  
    ~Stack() { }  
  
    void push(const int& item) // Has type int  
    { data_[++top_] = item; }  
  
    void pop()  
    { --top_; }  
  
    const int& top() const // Has type int  
    { return data_[top_]; }  
  
    bool empty() const  
    { return top_ == -1; }  
};
```

- Stack of int



Class Template

Module M39

Partha Pratim Das

Objectives & Outlines

What is a Template?

Function Template

Class Template

Definition

Instantiation

Partial Template

Instantiation & Default Template Parameters

Inheritance

Module Summary

- A **class template**
 - describes how a class should be built
 - supplies the class description and the definition of the member functions using some arbitrary type name, (as a place holder)
 - is a:
 - ▷ **parameterized** type with
 - ▷ **parameterized** member functions
 - can be considered the definition for a **unbounded set** of class types
 - is identified by the keyword **template**
 - ▷ followed by comma-separated list of **parameter** identifiers (each preceded by keyword **class** or keyword **typename**)
 - ▷ enclosed between **<** and **>** delimiters
 - ▷ followed by the definition of the class
 - is often used for **container** classes
 - Note that every template parameter is a **built-in type** or **class** – type parameters



Stack as a Class Template: Stack.h

Module M39

Partha Pratim
Das

Objectives &
Outlines

What is a
Template?

Function
Template

Class Template

Definition

Instantiation

Partial Template

Instantiation &
Default Template
Parameters

Inheritance

Module Summary

```
template<class T>
class Stack {
    T data_[100];
    int top_;
public:
    Stack() :top_(-1) { }
    ~Stack() { }

    void push(const T& item) { data_[++top_] = item; }

    void pop() { --top_; }

    const T& top() const { return data_[top_]; }

    bool empty() const { return top_ == -1; }
};
```

- Stack of type variable T
- The traits of type variable T include
copy assignment operator (T operator=(const T&))
- We do not call our template class as stack because std namespace has a class stack



Reverse String: Using Stack template

Module M39

Partha Pratim
Das

Objectives &
Outlines

What is a
Template?

Function
Template

Class Template

Definition

Instantiation

Partial Template

Instantiation &
Default Template
Parameters

Inheritance

Module Summary

```
#include <iostream>
#include <cstring>
using namespace std;

#include "Stack.h"

int main() {
    char str[10] = "ABCDE";

    Stack<char> s;          // Instantiated for char

    for (unsigned int i = 0; i < strlen(str); ++i)
        s.push(str[i]);

    cout << "Reversed String: ";
    while (!s.empty()) {
        cout << s.top();
        s.pop();
    }

    return 0;
}
```

- Stack of type char



Postfix Expression Evaluation: Using Stack template

Module M39

Partha Pratim Das

Objectives & Outlines

What is a Template?

Function Template

Class Template

Definition

Instantiation

Partial Template

Instantiation & Default Template Parameters

Inheritance

Module Summary

```
#include <iostream>
#include "Stack.h"
using namespace std;

int main() { // Postfix expression: 1 2 3 * + 9 -
    unsigned int postfix[] = { '1', '2', '3', '*', '+', '9', '-' }, ch;

    Stack<int> s;          // Instantiated for int

    for (unsigned int i = 0; i < sizeof(postfix) / sizeof(unsigned int); ++i) {
        ch = postfix[i];
        if (isdigit(ch)) { s.push(ch - '0'); }
        else {
            int op1 = s.top(); s.pop();
            int op2 = s.top(); s.pop();
            switch (ch) {
                case '*': s.push(op2 * op1); break;
                case '/': s.push(op2 / op1); break;
                case '+': s.push(op2 + op1); break;
                case '-': s.push(op2 - op1); break;
            }
        }
    }

    cout << "\n Evaluation " << s.top();
}
```



Template Parameter Traits

Module M39

Partha Pratim
Das

Objectives &
Outlines

What is a
Template?

Function
Template

Class Template
Definition

Instantiation

Partial Template

Instantiation &
Default Template
Parameters

Inheritance

Module Summary

- Parameter Types
 - may be of any type (including user defined types)
 - may be parameterized types, (that is, templates)
 - MUST support the methods used by the template functions:
 - ▷ What are the required constructors?
 - ▷ The required operator functions?
 - ▷ What are the necessary defining operations?



Function Template Instantiation: RECAP (Module 38)

Module M39

Partha Pratim Das

Objectives & Outlines

What is a Template?

Function Template

Class Template Definition

Instantiation

Partial Template Instantiation & Default Template Parameters

Inheritance

Module Summary

- Each item in the template parameter list is a template argument
- When a template function is invoked, the values of the template arguments are determined by seeing the types of the function arguments

```
template<class T> T Max(T x, T y);  
template<> char *Max<char *>(char *x, char *y);  
template <class T, int size> T Max(T x[size]);  
  
int a, b; Max(a, b);           // Binds to Max<int>(int, int);  
double c, d; Max(c, d);       // Binds to Max<double>(double, double);  
char *s1, *s2; Max(s1, s2);   // Binds to Max<char*>(char*, char*);  
  
int pval[9]; Max<int, 7>(arr); // Binds to Max<int, n>(int[]);
```

- Three kinds of conversions are allowed
 - L-value transformation (for example, Array-to-pointer conversion)
 - Qualification conversion
 - Conversion to a base class instantiation from a class template
- If the same template parameter are found for more than one function argument, template argument deduction from each function argument must be the same



Class Template Instantiation

Module M39

Partha Pratim Das

Objectives & Outlines

What is a Template?

Function Template

Class Template

Definition

Instantiation

Partial Template

Instantiation & Default Template

Parameters

Inheritance

Module Summary

- Class Template is instantiated *only when it is required*:
 - `template<class T> class Stack;` // Is a forward declaration
 - `Stack<char> s;` // Is an error
 - `Stack<char> *ps;` // Is okay
 - `void ReverseString(Stack<char>& s, char *str);` Is okay
- Class template is instantiated before
 - An object is defined with class template instantiation
 - If a pointer or a reference is dereferenced (for example, a method is invoked)
- A template definition can refer to a class template or its instances but a non-template can only refer to template instances



Class Template Instantiation Example

Module M39

Partha Pratim Das

Objectives & Outlines

What is a Template?

Function Template

Class Template

Definition

Instantiation

Partial Template

Instantiation &

Default Template

Parameters

Inheritance

Module Summary

```
#include <iostream>
#include <cstring>
using namespace std;
template<class T> class Stack;           // Forward declaration
void ReverseString(Stack<char>& s, char *str); // Stack template definition is not needed

template<class T>                       // Definition
class Stack { T data_[100]; int top_;
public: Stack() :top_(-1) { } ~Stack() { }
    void push(const T& item) { data_[++top_] = item; }
    void pop() { --top_; }
    const T& top() const { return data_[top_]; }
    bool empty() const { return top_ == -1; }
};

int main() { char str[10] = "ABCDE";
    Stack<char> s;                       // Stack template definition is needed
    ReverseString(s, str);
}

void ReverseString(Stack<char>& s, char *str) { // Stack template definition is needed
    for (unsigned int i = 0; i < strlen(str); ++i)
        s.push(str[i]);
    cout << "Reversed String: ";
    while (!s.empty())
        { cout << s.top(); s.pop(); }
}
```



Partial Template Instantiation and Default Template Parameters

Module M39

Partha Pratim
Das

Objectives &
Outlines

What is a
Template?

Function
Template

Class Template

Definition

Instantiation

Partial Template
Instantiation &
Default Template
Parameters

Inheritance

Module Summary

```
#include <iostream>
#include <string>
#include <cstring>
template<class T1 = int, class T2 = string> // Version 1 with default parameters
class Student { T1 roll_; T2 name_;
public: Student(T1 r, T2 n) : roll_(r), name_(n) { }
    void Print() const { std::cout << "Version 1: (" << name_ << ", " << roll_ << ")" << std::endl; }
};
template<class T1> // Version 2: Partial Template Specialization
class Student<T1, char *> { T1 roll_; char *name_;
public: Student(T1 r, char *n) : roll_(r), name_(std::strcpy(new char[std::strlen(n) + 1], n)) { }
    void Print() const { std::cout << "Version 2: (" << name_ << ", " << roll_ << ")" << std::endl; }
};
int main() {
    Student<int, string> s1(2, "Ramesh"); s1.Print(); // Version 1: T1 = int, T2 = string
    Student<int> s2(11, "Shampa"); s2.Print(); // Version 1: T1 = int, defa T2 = string
    Student<> s3(7, "Gagan"); s3.Print(); // Version 1: defa T1 = int, defa T2 = string
    Student<string> s4("X9", "Lalita"); s4.Print(); // Version 1: T1 = string, defa T2 = string
    Student<int, char*> s5(3, "Gouri"); s5.Print(); // Version 2: T1 = int, T2 = char*
}

Version 1: (Ramesh, 2)
Version 1: (Shampa, 11)
Version 1: (Gagan, 7)
Version 1: (Lalita, X9)
Version 2: (Gouri, 3)
```




Templates and Inheritance: Example (List.h)

Module M39

Partha Pratim
Das

Objectives &
Outlines

What is a
Template?

Function
Template

Class Template

Definition

Instantiation

Partial Template
Instantiation &
Default Template
Parameters

Inheritance

Module Summary

```
#ifndef __LIST_H
#define __LIST_H

#include <vector>
using namespace std;

template<class T>
class List {
public:
    void put(const T &val) { items.push_back(val); }
    int length() { return items.size(); }           // vector<T>::size()
    bool find(const T &val) {
        for (unsigned int i = 0; i < items.size(); ++i)
            if (items[i] == val) return true;      // T must support operator==(). Its trait
        return false;
    }
private:
    vector<T> items;                               // T must support T(), ~T(), T(const t&) or move
};                                                  // Its traits

#endif // __LIST_H
```

- List is basic container class



Templates and Inheritance: Example (Set.h)

Module M39

Partha Pratim
Das

Objectives &
Outlines

What is a
Template?

Function
Template

Class Template

Definition

Instantiation

Partial Template
Instantiation &
Default Template
Parameters

Inheritance

Module Summary

```
#ifndef __SET_H
#define __SET_H
#include "List.h"

template<class T>
class Set { public:
    Set() { };
    virtual ~Set() { };
    virtual void add(const T &val);
    int length(); // List<T>::length()
    bool find(const T &val); // List<T>::find()
private:
    List<T> items; // Container List<T>
};

template<class T>
void Set<T>::add(const T &val) {
    if (items.find(val)) return; // Don't allow duplicate
    items.put(val);
}

template<class T> int Set<T>::length() { return items.length(); }
template<class T> bool Set<T>::find(const T &val) { return items.find(val); }
#endif // __SET_H
```

- Set is a base class for a set
- Set uses List for container



Templates and Inheritance: Example (BoundSet.h)

Module M39

Partha Pratim
Das

Objectives &
Outlines

What is a
Template?

Function
Template

Class Template

Definition

Instantiation

Partial Template

Instantiation &
Default Template

Parameters

Inheritance

Module Summary

```
#ifndef __BOUND_SET_H
#define __BOUND_SET_H

#include "Set.h"

template<class T>
class BoundSet: public Set<T> {
public:
    BoundSet(const T &lower, const T &upper);
    void add(const T &val); // add() overridden to check bounds
private:
    T min;
    T max;
};

template<class T> BoundSet<T>::BoundSet(const T &lower, const T &upper): min(lower), max(upper) { }
template<class T> void BoundSet<T>::add(const T &val) {
    if (find(val)) return; // Set<T>::find()
    if ((val <= max) && (val >= min)) // T must support operator<=() and operator>=(). Its trait
        Set<T>::add(val); // Uses add() from parent class
}
#endif // __BOUND_SET_H
```

- BoundSet is a specialization of Set
- BoundSet is a set of bounded items



Templates and Inheritance: Example (Bounded Set Application)

Module M39

Partha Pratim
Das

Objectives &
Outlines

What is a
Template?

Function
Template

Class Template

Definition

Instantiation

Partial Template

Instantiation &
Default Template

Parameters

Inheritance

Module Summary

```
#include <iostream>
using namespace std;
#include "BoundSet.h"

int main() {
    BoundSet<int> bsi(3, 21);           // Allow values between 3 and 21
    Set<int> *setptr = &bsi;

    for (int i = 0; i < 25; i++)
        setptr->add(i);                // Set<T>::add(const T&) is virtual

    if (bsi.find(4))                    // Within bound
        cout << "We found an expected value\n";
    if (!bsi.find(0))                   // Outside lower bound
        cout << "We found NO unexpected value\n";
    if (!bsi.find(25))                  // Outside upper bound
        cout << "We found NO unexpected value\n";
}
```

```
We found an expected value
We found NO unexpected value
We found NO unexpected value
```

- Uses BoundSet to maintain and search elements



Module Summary

Module M39

Partha Pratim
Das

Objectives &
Outlines

What is a
Template?

Function
Template

Class Template

Definition

Instantiation

Partial Template

Instantiation &
Default Template
Parameters

Inheritance

Module Summary

- Introduced the templates in C++
- Discussed class templates as generic solution for data structure reuse
- Explained partial template instantiation and default template parameters
- Demonstrated templates on inheritance hierarchy
- Illustrated with examples



Module M40

Partha Pratim
Das

Objectives &
Outlines

Callable Entities

Function Pointers

Replace Switch / IF
Statements

Late Binding

Virtual Function

Callback

qsort

Issues

Functors

Basic Functor

Simple Example

Examples from STL

Function Pointer

Functor w/o state

Functor w/ state

Module Summary

Programming in Modern C++

Module M40: Functors: Function Objects

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Module Recap

Module M40

Partha Pratim Das

Objectives & Outlines

Callable Entities

Function Pointers

Replace Switch / IF Statements

Late Binding

Virtual Function

Callback

qsort

Issues

Functors

Basic Functor

Simple Example

Examples from STL

Function Pointer

Functor w/o state

Functor w/ state

Module Summary

- Discussed class templates as generic solution for data structure reuse
- Explained partial template instantiation and default template parameters
- Demonstrated templates on inheritance hierarchy
- Illustrated with examples



Module Objectives

Module M40

Partha Pratim
Das

Objectives & Outlines

Callable Entities

Function Pointers

Replace Switch / IF
Statements

Late Binding

Virtual Function

Callback

qsort

Issues

Functors

Basic Functor

Simple Example

Examples from STL

Function Pointer

Functor w/o state

Functor w/ state

Module Summary

- Understand the Function Objects or Functor
- Study the utility of functor in design, especially in STL



Module Outline

Module M40

Partha Pratim
Das

Objectives & Outlines

Callable Entities

Function Pointers

Replace Switch / IF
Statements

Late Binding

Virtual Function

Callback

qsort

Issues

Functors

Basic Functor

Simple Example

Examples from STL

Function Pointer

Functor w/o state

Functor w/ state

Module Summary

1 Callable Entities

2 Function Pointers

- Replace Switch / IF Statements
- Late Binding
- Virtual Function
- Callback
 - qsort
- Issues

3 Functors in C++

- Basic Functor
- Simple Example
- Examples from STL
 - Function Pointer
 - Functor without state
 - Functor with state

4 Module Summary

Programming in Modern C++

Partha Pratim Das

M40.4



Callable Entities

Module M40

Partha Pratim
Das

Objectives &
Outlines

Callable Entities

Function Pointers

Replace Switch / IF
Statements

Late Binding

Virtual Function

Callback

qsort

Issues

Functors

Basic Functor

Simple Example

Examples from STL

Function Pointer

Functor w/o state

Functor w/ state

Module Summary

Callable Entities



Callable Entities in C / C++

Module M40

Partha Pratim Das

Objectives & Outlines

Callable Entities

Function Pointers

Replace Switch / IF Statements

Late Binding

Virtual Function

Callback

qsort

Issues

Functors

Basic Functor

Simple Example

Examples from STL

Function Pointer

Functor w/o state

Functor w/ state

Module Summary

- A Callable Entity is an object that
 - Can be called using the function call syntax
 - Supports *Function Call Operator*: `operator()`
- Such objects are often called
 - A *Function Object* or
 - A *Functor*

Functors

Some authors distinguish between *Callable Entities*, *Function Objects* and *Functors*, but we will treat these terminology equivalently depending on the context



Several Callable Entities C++

Module M40

Partha Pratim Das

Objectives &
Outlines

Callable Entities

Function Pointers

Replace Switch / IF
Statements

Late Binding

Virtual Function

Callback

qsort

Issues

Functors

Basic Functor

Simple Example

Examples from STL

Function Pointer

Functor w/o state

Functor w/ state

Module Summary

- *Function-like Macros*
- *C Functions* (Global or in Namespace)
- *Member Functions*
 - Static
 - Non-Static
- *Pointers to Functions*
 - C Functions
 - Member Functions (static / Non-Static)
- *References to functions*: Acts like const pointers to functions
- *Functors*: Objects that define `operator()`



Function Pointers

Module M40

Partha Pratim
Das

Objectives &
Outlines

Callable Entities

Function Pointers

Replace Switch / IF
Statements

Late Binding

Virtual Function

Callback

qsort

Issues

Functors

Basic Functor

Simple Example

Examples from STL

Function Pointer

Functor w/o state

Functor w/ state

Module Summary

Function Pointers



Function Pointers

Module M40

Partha Pratim Das

Objectives & Outlines

Callable Entities

Function Pointers

Replace Switch / IF Statements

Late Binding

Virtual Function

Callback

qsort

Issues

Functors

Basic Functor

Simple Example

Examples from STL

Function Pointer

Functor w/o state

Functor w/ state

Module Summary

- *Points to the address of a function*
 - Ordinary C functions
 - Static C++ member functions
 - Non-static C++ member functions
- *Points to a function with a specific signature*
 - List of Calling Parameter Types
 - Return-Type
 - Calling Convention



Function Pointers in C

Module M40

Partha Pratim
Das

Objectives &
Outlines

Callable Entities

Function Pointers

Replace Switch / IF
Statements

Late Binding

Virtual Function

Callback

qsort

Issues

Functors

Basic Functor

Simple Example

Examples from STL

Function Pointer

Functor w/o state

Functor w/ state

Module Summary

- *Define a Function Pointer*

```
int (*pt2Function) (int, char, char);
```

- *Calling Convention*

```
int DoIt (int a, char b, char c); // __cdecl, __stdcall used in MSVC
int DoIt (int a, char b, char c) {
    printf ("DoIt\n");
    return a+b+c;
}
```

- *Assign Address to a Function Pointer*

```
pt2Function = &DoIt; // OR
pt2Function = DoIt;
```

- *Compare Function Pointers*

```
if (pt2Function == &DoIt) {
    printf ("pointer points to DoIt\n");
}
```

- *Call the Function pointed by the Function Pointer*

```
int result = (*pt2Function) (12, 'a', 'b');
```



Function Pointers in C

Module M40

Partha Pratim Das

Objectives & Outlines

Callable Entities

Function Pointers

Replace Switch / IF Statements

Late Binding

Virtual Function

Callback

qsort

Issues

Functors

Basic Functor

Simple Example

Examples from STL

Function Pointer

Functor w/o state

Functor w/ state

Module Summary

Direct Function Pointer

```
#include <stdio.h>

int (*pt2Function) (int, char, char);
int DoIt (int a, char b, char c);

int main() {
    pt2Function = DoIt; // &DoIt

    int result = (*pt2Function)(12, 'a', 'b');

    printf("%d", result);

    return 0;
}

int DoIt (int a, char b, char c) {
    printf ("DoIt\n");

    return a + b + c;
}
```

DoIt
207

Using typedef

```
#include <stdio.h>

typedef int (*pt2Function) (int, char, char);
int DoIt (int a, char b, char c);

int main() {
    pt2Function f = &DoIt; // DoIt

    int result = f(12, 'a', 'b');

    printf("%d", result);

    return 0;
}

int DoIt (int a, char b, char c) {
    printf ("DoIt\n");

    return a + b + c;
}
```

DoIt
207



Function Reference In C++

Module M40

Partha Pratim Das

Objectives & Outlines

Callable Entities

Function Pointers

Replace Switch / IF Statements

Late Binding

Virtual Function

Callback

qsort

Issues

Functors

Basic Functor

Simple Example

Examples from STL

Function Pointer

Functor w/o state

Functor w/ state

Module Summary

- *Define a Function Pointer*

```
int (A::*pt2Member)(float, char, char);
```

- *Calling Convention*

```
class A {  
    int DoIt (float a, char b, char c) {  
        cout << "A::DoIt" << endl; return a+b+c; }  
};
```

- *Assign Address to a Function Pointer*

```
pt2Member = &A::DoIt;
```

- *Compare Function Pointers*

```
if (pt2Member == &A::DoIt) {  
    cout <<"pointer points to A::DoIt" << endl;  
}
```

- *Call the Function pointed by the Function Pointer*

```
int result = (*this.*pt2Member)(12, 'a', 'b');
```



Function Pointer: Operations and Programming Techniques

Module M40

Partha Pratim Das

Objectives & Outlines

Callable Entities

Function Pointers

Replace Switch / IF Statements

Late Binding

Virtual Function

Callback

qsrt

Issues

Functors

Basic Functor

Simple Example

Examples from STL

Function Pointer

Functor w/o state

Functor w/ state

Module Summary

• Operations

- *Assign* an Address to a Function Pointer
- *Compare* two Function Pointers
- *Call* a Function using a Function Pointer
- *Pass* a Function Pointer as an Argument
- *Return* a Function Pointer
- *Arrays* of Function Pointers

• Programming Techniques

- *Replacing switch/if-statements*
- *Realizing user-defined late-binding*, or
 - ▷ Functions in Dynamically Loaded Libraries
 - ▷ Virtual Functions
- *Implementing callbacks*



Function Pointers: Replace Switch/ IF Statements

Module M40

Partha Pratim
Das

Objectives &
Outlines

Callable Entities

Function Pointers

Replace Switch / IF
Statements

Late Binding

Virtual Function

Callback

qsort

Issues

Functors

Basic Functor

Simple Example

Examples from STL

Function Pointer

Functor w/o state

Functor w/ state

Module Summary

Solution Using switch

```
#include <iostream>
using namespace std ;
// The four arithmetic operations
float Plus(float a, float b) { return a+b ; }
float Minus(float a, float b) { return a-b ; }
float Multiply(float a, float b) { return a*b ; }
float Divide(float a, float b) { return a/b ; }
void Switch(float a, float b, char opCode) {
    float result;
    switch (opCode) { // execute operation
        case '+': result = Plus(a, b); break;
        case '-': result = Minus(a, b); break;
        case '*': result = Multiply(a, b); break;
        case '/': result = Divide(a, b); break;
    }
    cout << "Result of = " << result << endl;
}

int main() { float a = 10.5, b = 2.5 ;
    Switch(a, b, '+');
    Switch(a, b, '-');
    Switch(a, b, '*');
    Switch(a, b, '/');
    return 0;
}
```

Solution Using Function Pointer

```
#include <iostream>
using namespace std ;
// The four arithmetic operations
float Plus(float a, float b)
{ return a+b; }
float Minus(float a, float b)
{ return a-b; }
float Multiply(float a, float b)
{ return a*b; }
float Divide(float a, float b)
{ return a/b; }
// Solution with Function pointer
void Switch (float a, float b,
    float (*pt2Func)(float, float)) {
    float result = pt2Func(a, b);
    cout << "Result := " << result << endl;
}

int main() { float a = 10.5, b = 2.5 ;
    Switch(a, b, &Plus);
    Switch(a, b, &Minus);
    Switch(a, b, &Multiply);
    Switch(a, b, &Divide);
    return 0;
}
```



Function Pointers: Late Binding / Dynamically Loaded Library

- A C Feature in Shared Dynamically Loaded Libraries

Program Part-1

```
#include <dlfcn.h>

int main() {
    void* handle = dlopen("hello.so", RTLD_LAZY);
    typedef void (*hello_t)();

    hello_t myHello = 0;
    myHello = (hello_t)dlsym(handle, "hello");
    myHello();

    dlclose(handle);
}
```

Program Part-2

```
#include <iostream>
using namespace std;

extern "C" void hello() {
    cout << "hello" << endl;
}
```



Function Pointers: Late Binding / Virtual Function

Module M40

Partha Pratim Das

Objectives & Outlines

Callable Entities

Function Pointers

Replace Switch / IF Statements

Late Binding

Virtual Function

Callback

qsort

Issues

Functors

Basic Functor

Simple Example

Examples from STL

Function Pointer

Functor w/o state

Functor w/ state

Module Summary

- A C++ Feature for Polymorphic Member Functions

Code Snippet Part-1

```
class A {  
    public:  
        void f();  
        virtual void g();  
};  
  
class B: public A {  
    public:  
        void f();  
        virtual void g();  
};
```

Code Snippet Part-2

```
int main() {  
    A a;  
    B b;  
    A *p = &b;  
  
    a.f(); // A::f()  
    a.g(); // A::g()  
    p->f(); // A::f()  
    p->g(); // B::g()  
}
```



Example: Callback, Function Pointers

Module M40

Partha Pratim
Das

Objectives &
Outlines

Callable Entities

Function Pointers

Replace Switch / IF
Statements

Late Binding

Virtual Function

Callback

qsort

Issues

Functors

Basic Functor

Simple Example

Examples from STL

Function Pointer

Functor w/o state

Functor w/ state

Module Summary

- It is a Common C Feature

```
// Application
extern void (*func)();
void f() { }
int main() {
    func = &f;
    g();
}

// Library
void (*func)();
void g() {
    (*func)();
}
```



Function Pointers: Callback Illustration (Step 1)

Module M40

Partha Pratim Das

Objectives & Outlines

Callable Entities

Function Pointers

Replace Switch / IF Statements

Late Binding

Virtual Function

Callback

qsort

Issues

Functors

Basic Functor

Simple Example

Examples from STL

Function Pointer

Functor w/o state

Functor w/ state

Module Summary

```
// Application
extern void (*func) ();
void f()
{

}

void main()
{
    func = &f;

    g();
}
```

```
// Library
void (*func) ();

void g()
{

    (*func) ();

}
```



Function Pointers: Callback Illustration (Step 2)

Module M40

Partha Pratim Das

Objectives & Outlines

Callable Entities

Function Pointers

Replace Switch / IF Statements

Late Binding

Virtual Function

Callback

qsort

Issues

Functors

Basic Functor

Simple Example

Examples from STL

Function Pointer

Functor w/o state

Functor w/ state

Module Summary

```
// Application
extern void (*func) ();
void f()
{
}

void main()
{
    func = &f;

    g();
}
```

```
// Library
void (*func) ();

void g()
{
    (*func) ();
}
```




Function Pointers: Callback Illustration (Step 3)

Module M40

Partha Pratim Das

Objectives & Outlines

Callable Entities

Function Pointers

Replace Switch / IF Statements

Late Binding

Virtual Function

Callback

qsort

Issues

Functors

Basic Functor

Simple Example

Examples from STL

Function Pointer

Functor w/o state

Functor w/ state

Module Summary

```
// Application
extern void (*func) ();
void f()
{
}

void main()
{
    func = &f;

    g();
}
```

```
// Library
void (*func) ();

void g()
{
    (*func) ();
}
```



Function Pointers: Callback Illustration (Step 4)

Module M40

Partha Pratim Das

Objectives & Outlines

Callable Entities

Function Pointers

Replace Switch / IF Statements

Late Binding

Virtual Function

Callback

qsort

Issues

Functors

Basic Functor

Simple Example

Examples from STL

Function Pointer

Functor w/o state

Functor w/ state

Module Summary

```
// Application
extern void (*func)();
void f()
{
    Callback
}

void main()
{
    func = &f;

    g();
}
```

```
// Library
void (*func)();

void g()
{
    (*func)();
}
```



Function Pointers: Callback Illustration (Step-Final)

Module M40

Partha Pratim Das

Objectives & Outlines

Callable Entities

Function Pointers

Replace Switch / IF Statements

Late Binding

Virtual Function

Callback

qsort

Issues

Functors

Basic Functor

Simple Example

Examples from STL

Function Pointer

Functor w/o state

Functor w/ state

Module Summary

```
// Application
extern void (*func) ();
void f()
{

}

void main()
{
    func = &f;

    g();
}
```

```
// Library
void (*func) ();

void g()
{

    (*func) ();
}
```



Function Pointers: Callback Illustration: Whole Process

Module M40

Partha Pratim Das

Objectives & Outlines

Callable Entities

Function Pointers

Replace Switch / IF Statements

Late Binding

Virtual Function

Callback

qsort

Issues

Functors

Basic Functor

Simple Example

Examples from STL

Function Pointer

Functor w/o state

Functor w/ state

Module Summary

```
// Application
extern void (*func)();
void f()
{
}

void main()
{
    func = &f;
    g();
}
```

```
// Library
void (*func)();

void g()
{
    (*func)();
}
```

Callback

Step-2

Step-3

Step-1

Step-4



Function Pointers: Callback: qsort to Quick Sort

Module M40

Partha Pratim
Das

Objectives &
Outlines

Callable Entities

Function Pointers

Replace Switch / IF
Statements

Late Binding

Virtual Function

Callback

qsort

Issues

Functors

Basic Functor

Simple Example

Examples from STL

Function Pointer

Functor w/o state

Functor w/ state

Module Summary

```
void qsort(void *base,    // Pointer to the first element of the array to be sorted
           size_t nitems, // Number of elements in the array pointed by base
           size_t size,   // Size in bytes of each element in the array
           int (*compar)(const void *, const void*)); // Function that compares two elements

int CmpFunc(const void* a, const void* b) { // Compare function for int
    int ret = (*(const int*)a > *(const int*) b)? 1:
              (*(const int*)a == *(const int*) b)? 0: -1;
    return ret;
}

int main() {
    int field[10];

    for(int c = 10; c>0; c--)
        field[10-c] = c;

    qsort((void*) field, 10, sizeof(field[0]), CmpFunc);
}
```



Function Pointers: Issues

Module M40

Partha Pratim
Das

Objectives &
Outlines

Callable Entities

Function Pointers

Replace Switch / IF
Statements

Late Binding

Virtual Function

Callback

qsort

Issues

Functors

Basic Functor

Simple Example

Examples from STL

Function Pointer

Functor w/o state

Functor w/ state

Module Summary

- No value semantics
- Weak type checking
- Two function pointers having identical signature are necessarily indistinguishable
- No encapsulation for parameters



Functors in C++

Module M40

Partha Pratim
Das

Objectives &
Outlines

Callable Entities

Function Pointers

Replace Switch / IF
Statements

Late Binding

Virtual Function

Callback

qsort

Issues

Functors

Basic Functor

Simple Example

Examples from STL

Function Pointer

Functor w/o state

Functor w/ state

Module Summary

Functors in C++



Functors or Function Objects

Module M40

Partha Pratim Das

Objectives & Outlines

Callable Entities

Function Pointers

Replace Switch / IF Statements

Late Binding

Virtual Function

Callback

qsort

Issues

Functors

Basic Functor

Simple Example

Examples from STL

Function Pointer

Functor w/o state

Functor w/ state

Module Summary

- Smart Functions
 - Functors are *functions with a state*
 - Functors *encapsulate C / C++ function pointers*
 - ▷ Uses templates and
 - ▷ Engages polymorphism
- Has its own *Type*
 - A class with zero or more private members to store the state and an overloaded *operator()* to execute the function
- Usually *faster* than ordinary Functions
- Can be used to implement *callbacks*
- Provides the basis for *Command Design Pattern*



Basic Functor

Module M40

Partha Pratim Das

Objectives & Outlines

Callable Entities

Function Pointers

Replace Switch / IF Statements

Late Binding

Virtual Function

Callback

qsort

Issues

Functors

Basic Functor

Simple Example

Examples from STL

Function Pointer

Functor w/o state

Functor w/ state

Module Summary

- Any class that overloads the function call operator:
 - `void operator()();`
 - `int operator()(int, int);`
 - `double operator()(int, double);`
 - `...`



Functors: Simple Example

Module M40

Partha Pratim Das

Objectives & Outlines

Callable Entities

Function Pointers

Replace Switch / IF Statements

Late Binding

Virtual Function

Callback

qsort

Issues

Functors

Basic Functor

Simple Example

Examples from STL

Function Pointer

Functor w/o state

Functor w/ state

Module Summary

- Consider the code below

```
int AdderFunction(int a, int b) { // A function
    return a + b;
}

class AdderFunctor {
public:
    int operator()(int a, int b) { // A functor
        return a + b;
    }
};

int main() {
    int x = 5;
    int y = 7;
    int z = AdderFunction(x, y); // Function invocation

    AdderFunctor aF;
    int w = aF(x, y);           // aF.operator()(x, y); -- Functor invocation
}
```



Functors: Examples from STL: Function Pointer for Functor

Module M40

Partha Pratim Das

Objectives & Outlines

Callable Entities

Function Pointers

Replace Switch / IF Statements

Late Binding

Virtual Function

Callback

qsort

Issues

Functors

Basic Functor

Simple Example

Examples from STL

Function Pointer

Functor w/o state

Functor w/ state

Module Summary

- **Fill a vector with random numbers**

- **generate** algorithm

```
#include <algorithm>
template <class ForwardIterator, class Generator>
    void generate(ForwardIterator first, ForwardIterator last, Generator gen) {
        while (first != last) {
            *first = gen();
            ++first;
        }
    }
```

- ▷ **first, last:** Iterators are defined for a range in the sequence. "[" or "]" means **include** the element and "(" or ")" means **exclude** the element. **ForwardIterator has a range [first,last)** spanning from first element to the element before the last
 - ▷ **gen:** Generator function that is called with no arguments and returns some value of a type convertible to those pointed by the iterators
 - ▷ This can either be a **function pointer** or a **function object**

- Function Pointer **rand** as Function Object

```
#include <cstdlib>

// int rand (void);

vector<int> V(100);
generate(V.begin(), V.end(), rand);
```



Functors: Examples from STL: Functor without a state

Module M40

Partha Pratim Das

Objectives & Outlines

Callable Entities

Function Pointers

Replace Switch / IF Statements

Late Binding

Virtual Function

Callback

qsort

Issues

Functors

Basic Functor

Simple Example

Examples from STL

Function Pointer

Functor w/o state

Functor w/ state

Module Summary

- **Sort a vector of double by magnitude**

- **sort** algorithm

```
#include <algorithm>
template <class RandomAccessIterator, class Compare>
    void sort (RandomAccessIterator first, // Simple interface
              RandomAccessIterator last,  // Difficult to use incorrectly
              Compare comp); // Compare Functor
```

- ▷ **first, last:** `RandomAccessIterator` has a range `[first,last)`
- ▷ `RandomAccessIterator` shall point to a type for which swap is properly defined and which is both `move-constructible` and `move-assignable` (C++11)
- ▷ **comp:** Binary function that accepts two elements in the range as arguments, and returns a value convertible to bool. The value returned indicates whether the element passed as first argument is considered to go before the second in the specific strict weak ordering it defines.
- ▷ The function shall not modify any of its arguments
- ▷ This can either be a `function pointer` or a `function object`



Functors: Examples from STL: Functor without a state

- Sort a vector of double by magnitude

Using `qsort` in C with User-defined **Function** `less_mag`

```
#include <stdlib.h>
// Compare Function pointer
void qsort(void *base,
           size_t nitems,
           size_t size,
           int (*compar)(const void *, const void*))
// Complicated interface. Difficult to use correctly

// Type-unsafe comparison function
// Intricate and error-prone with void*
int less_mag(const void* a, const void* b) {
    return (fabs(*(const double*)a) <
           fabs(*(const double*)b) ? 1: 0;
}

double V[100]; // Capacity = 100
// 10 elements are filled - needs to be tracked

// Difficult to call
qsort((void*) V, 10, sizeof(V[0]), less_mag);
```

Using `sort` in C++ with User-defined **Functor** `less_mag`

```
#include <algorithm>
// Compare Functor
template <class RandomAccessIterator, class Compare>
void sort (RandomAccessIterator first,
           RandomAccessIterator last,
           Compare comp);
// Simple interface. Difficult to use incorrectly

// Type-safe comparison functor
struct less_mag: public
    binary_function<double, double, bool> {
    bool operator()(double x, double y)
    { return fabs(x) < fabs(y); }
};

vector<double> V(100);
// 10 elements are filled tracked automatically

// Easy to call
sort(V.begin(), V.end(), less_mag());
```



Functors: Examples from STL: Functor with a state

Module M40

Partha Pratim Das

Objectives & Outlines

Callable Entities

Function Pointers

Replace Switch / IF Statements

Late Binding

Virtual Function

Callback

qsort

Issues

Functors

Basic Functor

Simple Example

Examples from STL

Function Pointer

Functor w/o state

Functor w/ state

Module Summary

• Compute the sum of elements in a vector

○ `for_each` algorithm

```
#include <algorithm>
template<class InputIterator, class Function>
Function for_each(InputIterator first, InputIterator last, Function fn) {
    while (first!=last) {
        fn (*first);
        ++first;
    }
    return fn;          // or, since C++11: return move(fn);
}
```

▷ **first, last:** `InputIterator` has a range `[first,last)`

▷ **fn:** Unary function that accepts an element in the range as argument

▷ This can either be a function pointer or a `move constructible function object` (C++11)

▷ Its return value, if any, is ignored.

○ User-defined Functor `adder` with local state

```
struct adder: public unary_function<double, void> { adder() : sum(0) { }
    double sum; // Local state
    void operator()(double x) { sum += x; }
};
```

```
vector<double> V;
```

```
...
```

```
adder result = for_each(V.begin(), V.end(), adder());
```

```
cout << "The sum is " << result.sum << endl;
```



Module Summary

Module M40

Partha Pratim
Das

Objectives &
Outlines

Callable Entities

Function Pointers

Replace Switch / IF
Statements

Late Binding

Virtual Function

Callback

qsort

Issues

Functors

Basic Functor

Simple Example

Examples from STL

Function Pointer

Functor w/o state

Functor w/ state

Module Summary

- Introduced Function Objects or Functors
- Illustrated functors with several simple examples and examples from STL



Tutorial T08

Partha Pratim
Das

Tutorial Recap

Objective &
Outline

Int<N> UDT

Design

Operations

Class Design

Implementation

Test

Polynomial UDT

Design

Implementation

Test

Practice UDTs

Tutorial Summary

Programming in Modern C++

Tutorial T08: How to design a UDT like built-in types?: Part 2: Int and Poly UDT

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Tutorial Recap

Tutorial T08

Partha Pratim
Das

Tutorial Recap

Objective &
Outline

Int<N> UDT

Design

Operations

Class Design

Implementation

Test

Polynomial UDT

Design

Implementation

Test

Practice UDTs

Tutorial Summary

- Analysed the difference between Built-in & UDT
- Discussed the meaning of Building a data type
- Understood the necessity of Building a data type
- Built a Fraction data type by iterative refinement



Tutorial Objectives

Tutorial T08

Partha Pratim
Das

Tutorial Recap

Objective &
Outline

`Int<N>` UDT

Design

Operations

Class Design

Implementation

Test

Polynomial UDT

Design

Implementation

Test

Practice UDTs

Tutorial Summary

- To build more UDTs: `Int<N>` and `Poly<T>`
- To test mix of UDTs



Tutorial Outline

Tutorial T08

Partha Pratim Das

Tutorial Recap

Objective &
Outline

Int<N> UDT

Design

Operations

Class Design

Implementation

Test

Polynomial UDT

Design

Implementation

Test

Practice UDTs

Tutorial Summary

- 1 Tutorial Recap
- 2 Int<N> UDT
 - Design
 - Operations
 - Class Design
 - Implementation
 - Test
- 3 Polynomial UDT
 - Design
 - Implementation
 - Test
- 4 Practice UDTs
- 5 Tutorial Summary



Int<N> UDT

Tutorial T08

Partha Pratim
Das

Tutorial Recap

Objective &
Outline

Int<N> UDT

Design

Operations

Class Design

Implementation

Test

Polynomial UDT

Design

Implementation

Test

Practice UDTs

Tutorial Summary

Int<N> UDT



Understanding int

Tutorial T08

Partha Pratim Das

Tutorial Recap

Objective & Outline

Int<N> UDT

Design

Operations

Class Design

Implementation

Test

Polynomial UDT

Design

Implementation

Test

Practice UDTs

Tutorial Summary

- The datatype we are most familiar with is `int` which is a signed integer
 - Represented in a given number of bits, typically, 8, 16, 32, 64, or 128
 - Hence, `int` can represent numbers from `INT_MAX` to `INT_MIN`
 - For example, for 32 bits, $INT_MAX = 2^{31} - 1 = 2147483647$ and $INT_MIN = -2^{31} = -2147483648$
 - Beyond the `INT_MAX` .. `INT_MIN` range we get integer overflow and numbers wraparound

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    cout << INT_MAX    << endl; // 2147483647
    cout << INT_MIN    << endl; // -2147483648
    cout << INT_MAX+1  << endl; // -2147483648 integer overflow in expression of type 'int'
    cout << INT_MIN-1  << endl; // 2147483647 integer overflow in expression of type 'int'
    cout << -INT_MIN   << endl; // -2147483648 integer overflow in expression of type 'int'
}
```



Design of Int UDT

Tutorial T08

Partha Pratim
Das

Tutorial Recap

Objective &
Outline

Int<N> UDT

Design

Operations

Class Design

Implementation

Test

Polynomial UDT

Design

Implementation

Test

Practice UDTs

Tutorial Summary

- To understand `int` better, we intend to design a UDT `Int<N>` which can behave like `int` albeit for a of size $N > 0$ bits that can be specified
- The range of values will be:
 - $\text{MinInt} = -2^{N-1} \dots \text{MaxInt} = 2^{N-1} - 1$
- The broad tasks involved include:
 - Make a clear statement of the concept of `Int`
 - Identify a representation for a `Int` object
 - Identify the properties and assertions applicable to all objects
 - Identify the operations for `Int` objects
 - ▷ Choose appropriate operators to overload for the operations
 - ▷ For example `operator+` to add two `Int` objects, or `operator<<` to stream a `Int` to `cout`
 - ▷ *Do not break the natural semantics for the operators*



Notion of Int

Tutorial T08

Partha Pratim Das

Tutorial Recap

Objective & Outline

Int<N> UDT

Design

Operations

Class Design

Implementation

Test

Polynomial UDT

Design

Implementation

Test

Practice UDTs

Tutorial Summary

- Intuitively $\text{Int}\langle N \rangle$ is a notation for whole numbers of the form of N bits signed integers
- $\text{MinInt} = -2^{N-1} \dots \text{MaxInt} = 2^{N-1} - 1$
- Numbers in $\text{Int}\langle N \rangle$ bits within a range of values $\text{MinInt} .. \text{MaxInt}$. Beyond this range the numbers wrap around:
 - $\text{MaxInt} + 1 = \text{MinInt}$
 - $\text{MinInt} - 1 = \text{MaxInt}$
- For example:
 - $N = 4 \Rightarrow \text{Range: } -8 .. 7$
 - $\text{MinInt} = -2^3 = -8$ and $\text{MaxInt} = 2^3 - 1 = 7$
 - Except for *overflow*¹ as follows, all operations of $\text{Int}\langle N \rangle$ is same as `int`
 - ▷ $7 + 1 = -8$
 - ▷ $-8 - 1 = 7$
 - ▷ $-8 = -8$

¹Some authors distinguish between *overflow* (being more than MaxInt) and *underflow* (being less than MinInt). However, we prefer to use the term *overflow* in both cases because actually representation *overflows* the bits in both cases



Operations of $\text{Int}\langle N \rangle$

Tutorial T08

Partha Pratim Das

Tutorial Recap

Objective & Outline

$\text{Int}\langle N \rangle$ UDT

Design

Operations

Class Design

Implementation

Test

Polynomial UDT

Design

Implementation

Test

Practice UDTs

Tutorial Summary

Definition

Limits:

$$\text{MaxInt} = 2^{N-1} - 1$$

$$\text{MinInt} = -2^{N-1}$$

Negation:

$$\begin{aligned} -a &= a, \text{ if } a == \text{MinInt} \\ &= -a, \text{ otherwise} \end{aligned}$$

Addition:

$$\begin{aligned} a + b &= a + b - 2^N, \text{ if } a + b > \text{MaxInt} \\ &= a + b + 2^N, \text{ if } a + b < \text{MinInt} \\ &= a + b, \text{ otherwise} \end{aligned}$$

Subtraction:

$$a - b = a + (-b)$$

Let $N = 4$

Example 1: $2 + 3 = 5$. $4 + 7 = -5$. $(-5) + 6 = 1$. $(-3) + (-2) = -5$. $(-6) + -7 = 3$

Example 2: $2 - 3 = -1$. $4 - 7 = -3$. $(-5) - 6 = 5$. $(-3) - (-2) = -1$. $(-6) - (-7) = 1$

Multiplication, Division, and Modulus: Left as exercise



Design of `Int<N>` Class

Tutorial T08

Partha Pratim Das

Tutorial Recap

Objective & Outline

`Int<N>` UDT

Design

Operations

Class Design

Implementation

Test

Polynomial UDT

Design

Implementation

Test

Practice UDTs

Tutorial Summary

- Clearly, the representation of a `Int<N>` needs to be a template with an `unsigned int` param `N`
- For the implementation, the `Int<N>` needs to use an underlying type `T` where basic arithmetic operations are available. So `T` is a type parameter for `Int<N>`. By default this can be `int`
- It is important to note that `N <= sizeof(T) * 8`. Otherwise, our basic operations may overflow

- Hence, the `Int<N>` class would look like:

```
template<typename T = int, unsigned int N = 4>
class Int_ { // an N-bits integer class with underlying type T
    T v_;    // actual value in underlying type T
    // ... Rest of the class
}
```

- Note that we name the type as `Int_` so that we can conveniently alias it in the user program as:

```
template<typename T, unsigned int N> class Int_;
typedef Int_<int, 4> Int; // T = int and N = 4
// Use as Int
```

- `Int<N>` should support the operation of `int`:
- `Int<N>` should also support conversion the underlying type `T`
- `Int<N>` may support the following constants for convenience of implementation:

```
static const Int_<T, N> MaxInt; // 2^(N-1)-1
static const Int_<T, N> MinInt; // -2^(N-1)
```



Design of Int<N> Interface and Implementation

Tutorial T08

Partha Pratim Das

Tutorial Recap

Objective & Outline

Int<N> UDT

Design

Operations

Class Design

Implementation

Test

Polynomial UDT

Design

Implementation

Test

Practice UDTs

Tutorial Summary

```
template<typename T = int, unsigned int N = 4>
class Int_ { public:
    static const Int_<T, N> MaxInt; // 2^(N-1)-1
    static const Int_<T, N> MinInt; // -2^(N-1)

    explicit Int_<T, N>(int v = 1) : v_(v) { // Two overloads of Constructor
        assert(v_ <= static_cast<int>(MaxInt)); // assert will fire if the value is out of limits
        assert(v_ >= static_cast<int>(MinInt));
    }
    Int_<T, N>(const Int_<T, N>& i) : v_(i.v_) { } // Copy Constructor
    ~Int_<T, N>() { } // No virtual destructor needed
    Int_<T, N>& operator=(const Int_<T, N>& i) { v_ = i.v_; return *this; } // Assignment
    // Streaming operators for IO
    friend ostream& operator<<(ostream& os, const Int_<T, N>& i) { os << i.v_; return os; }
    friend istream& operator>>(istream& is, Int_<T, N>& i) {
        T v; is >> v; i = Int_<T, N>(v); // We deliberately construct to test that v is within limits
        return is;
    }
    // Unary arithmetic operators
    Int_<T, N> operator-() const { return Int_<T, N>(v_ == MinInt_T? v_: -v_); }
    Int_<T, N> operator+() const { return *this; }
    Int_<T, N>& operator++() { *this = *this + Int_<T, N>(1); return *this; }
    Int_<T, N>& operator--() { *this = *this - Int_<T, N>(1); return *this; }
    Int_<T, N> operator++(int) { Int_<T, N> i = *this; ++*this; return i; }
    Int_<T, N> operator--(int) { Int_<T, N> i = *this; --*this; return i; }
```



Design of Int<N> Interface and Implementation

Tutorial T08

Partha Pratim
Das

Tutorial Recap

Objective &
Outline

Int<N> UDT

Design

Operations

Class Design

Implementation

Test

Polynomial UDT

Design

Implementation

Test

Practice UDTs

Tutorial Summary

```
// Binary arithmetic operators
```

```
friend Int_<T, N> operator+(const Int_<T, N>& i1, const Int_<T, N>& i2) {  
    T v = i1.v_ + i2.v_; // add values in underlying type T  
    if (v > MaxInt_T) // MaxInt_T is MaxInt in type T  
        return Int_<T, N>(v - TwoPowerN_T); // wrap around if the value is more than MaxInt  
    else if (v < MinInt_T) // MinInt_T is MinInt in type T  
        return Int_<T, N>(v + TwoPowerN_T); // wrap around if the value is less than MinInt  
    else  
        return Int_<T, N>(v); // value within limits - no action  
}  
friend Int_<T, N> operator-(const Int_<T, N>& i1, const Int_<T, N>& i2) { return i1 + (-i2); }
```

```
// NOT IMPLEMENTED - left as exercises
```

```
friend Int_<T, N> operator*(const Int_<T, N>& i1, const Int_<T, N>& i2);  
friend Int_<T, N> operator/(const Int_<T, N>& i1, const Int_<T, N>& i2);  
friend Int_<T, N> operator%(const Int_<T, N>& i1, const Int_<T, N>& i2);
```

```
// Logical comparison operators
```

```
friend bool operator==(const Int_<T, N>& i1, const Int_<T, N>& i2) { return i1.v_ == i2.v_; }  
friend bool operator!=(const Int_<T, N>& i1, const Int_<T, N>& i2) { return i1.v_ != i2.v_; }  
friend bool operator<(const Int_<T, N>& i1, const Int_<T, N>& i2) { return i1.v_ < i2.v_; }  
friend bool operator<=(const Int_<T, N>& i1, const Int_<T, N>& i2) { return i1.v_ <= i2.v_; }  
friend bool operator>(const Int_<T, N>& i1, const Int_<T, N>& i2) { return i1.v_ > i2.v_; }  
friend bool operator>=(const Int_<T, N>& i1, const Int_<T, N>& i2) { return i1.v_ >= i2.v_; }
```



Design of Int<N> Interface and Implementation

Tutorial T08

Partha Pratim Das

Tutorial Recap

Objective & Outline

Int<N> UDT

Design

Operations

Class Design

Implementation

Test

Polynomial UDT

Design

Implementation

Test

Practice UDTs

Tutorial Summary

```
// Advanced assignment operators: NOT IMPLEMENTED - left as exercises
Int_<T, N>& operator+=(const Int_<T, N>& i);
Int_<T, N>& operator-=(const Int_<T, N>& i);
Int_<T, N>& operator*=(const Int_<T, N>& i);
Int_<T, N>& operator/=(const Int_<T, N>& i);
Int_<T, N>& operator%=(const Int_<T, N>& i);

operator T() const { return v_; } // conversion to underlying type T

private: // data members
    T v_; // Value in underlying type T
    static const T MaxInt_T; // MaxInt = 2^(N-1)-1 in underlying type T
    static const T MinInt_T; // MinInt = -2^(N-1) in underlying type T
    static const T TwoPowerN_T; // 2^N in underlying type T

public: static int Int_<T, N>::pow() { return Int_<T, N - 1>::pow() * 2; }
};
template<typename T> class Int_<T, 1> { public: static int Int_<T, 1>::pow() { return 1; } };

// Instantiations of static const members
const Int Int::MaxInt = Int(Int::pow() - 1);
const Int Int::MinInt = Int(-Int::pow());
const int Int::MaxInt_T = static_cast<int>(Int::MaxInt); // 2^(N-1)-1
const int Int::MinInt_T = static_cast<int>(Int::MinInt); // -2^(N-1)
const int Int::TwoPowerN_T = (Int::MaxInt_T+1) << 1; // 2^N
```

Programming in Modern C++ Partha Pratim Das



Testing Int<N>

Tutorial T08

Partha Pratim
Das

Tutorial Recap

Objective &
Outline

Int<N> UDT

Design

Operations

Class Design

Implementation

Test

Polynomial UDT

Design

Implementation

Test

Practice UDTs

Tutorial Summary

```
#include <iostream>
using namespace std;
```

```
template<typename T, unsigned int N> class Int_;
typedef Int_<int, 4> Int; // N = 4
```

```
#include "Int.h"
```

```
void main() {
    cout << "Int::MaxInt = " << Int::MaxInt << endl; // Int::MaxInt = 7
    cout << "Int::MinInt = " << Int::MinInt << endl; // Int::MinInt = -8
```

```
    // Constructor, Copy Operations and Write Test
```

```
    Int f1(5);    cout << "Int f1(5) = " << f1 << endl;    // Int f1(5) = 5
    Int f2(0);    cout << "Int f2(0) = " << f2 << endl;    // Int f2(0) = 0
    Int f3;       cout << "Int f3 = " << f3 << endl;       // Int f3 = 1
    Int f4(f1);   cout << "Int f4(f1) = " << f4 << endl;   // Int f4(f1) = 5
    cout << "Assignment: f2 = f1: f2 = " << (f2 = f1) << endl; // Assignment: f2 = f1: f2 = 5
```

```
    // Read Test
```

```
    cin >> f1;    // 3
    cout << "Read f1 = " << f1 << endl; // Read f1 = 3
```



Testing Int<N>

Tutorial T08

Partha Pratim Das

Tutorial Recap

Objective & Outline

Int<N> UDT

Design

Operations

Class Design

Implementation

Test

Polynomial UDT

Design

Implementation

Test

Practice UDTs

Tutorial Summary

// Unary Operations Test

```
cout << "-Int(2) = " << -Int(2) << endl;    // -Int(2) = -2
cout << "-Int(-2) = " << -Int(-2) << endl;    // -Int(-2) = 2
cout << "-Int(-8) = " << -Int(-8) << endl;    // -Int(-8) = -8
cout << "-Int(7) = " << -Int(7) << endl;      // -Int(7) = -7
cout << "++Int(2) = " << ++Int(2) << endl;    // ++Int(2) = 3
cout << "++Int(7) = " << ++Int(7) << endl;    // ++Int(7) = -8
cout << "++Int(-8) = " << ++Int(-8) << endl;  // ++Int(-8) = -7
cout << "--Int(0) = " << --Int(0) << endl;    // --Int(0) = -1
cout << "--Int(-7) = " << --Int(-7) << endl;  // --Int(-7) = -8
cout << "--Int(-8) = " << --Int(-8) << endl;  // --Int(-8) = 7
```

// Binary Operations Test

```
cout << "Int(2) + Int(3) = " << (Int(2) + Int(3)) << endl;    // Int(2) + Int(3) = 5
cout << "Int(4) + Int(7) = " << (Int(4) + Int(7)) << endl;    // Int(4) + Int(7) = -5
cout << "Int(-5) + Int(6) = " << (Int(-5) + Int(6)) << endl;  // Int(-5) + Int(6) = 1
cout << "Int(-3) + Int(-2) = " << (Int(-3) + Int(-2)) << endl; // Int(-3) + Int(-2) = -5
cout << "Int(-6) + Int(-7) = " << (Int(-6) + Int(-7)) << endl; // Int(-6) + Int(-7) = 3
```



Testing Int<N>

Tutorial T08

Partha Pratim
Das

Tutorial Recap

Objective & Outline

Int<N> UDT

Design

Operations

Class Design

Implementation

Test

Polynomial UDT

Design

Implementation

Test

Practice UDTs

Tutorial Summary

// Binary Operations Test

```
cout << "Int(2) - Int(3) = " << (Int(2) - Int(3)) << endl;    // Int(2) - Int(3) = -1
cout << "Int(4) - Int(7) = " << (Int(4) - Int(7)) << endl;    // Int(4) - Int(7) = -3
cout << "Int(-5) - Int(6) = " << (Int(-5) - Int(6)) << endl;   // Int(-5) - Int(6) = 5
cout << "Int(-3) - Int(-2) = " << (Int(-3) - Int(-2)) << endl; // Int(-3) - Int(-2) = -1
cout << "Int(-6) - Int(-7) = " << (Int(-6) - Int(-7)) << endl; // Int(-6) - Int(-7) = 1
```

// Logical Operations Test

```
cout << "Int(-6) == Int(-6): " << ((Int(-6) == Int(-6)) ? "T" : "F") << endl; // Int(-6) == Int(-6): T
cout << "Int(4) != Int(3): " << ((Int(4) != Int(3)) ? "T" : "F") << endl;    // Int(4) != Int(3): T
cout << "Int(-7) < Int(2): " << ((Int(-7) < Int(2)) ? "T" : "F") << endl;    // Int(-7) < Int(2): T
cout << "Int(-7) <= Int(2): " << ((Int(-7) <= Int(2)) ? "T" : "F") << endl;  // Int(-7) <= Int(2): T
cout << "Int(-5) > Int(-6): " << ((Int(-5) > Int(-6)) ? "T" : "F") << endl; // Int(-5) > Int(-6): T
cout << "Int(4) >= Int(4): " << ((Int(4) >= Int(4)) ? "T" : "F") << endl;    // Int(4) >= Int(4) = T
```

}



Polynomial UDT

Tutorial T08

Partha Pratim
Das

Tutorial Recap

Objective &
Outline

Int<N> UDT

Design

Operations

Class Design

Implementation

Test

Polynomial UDT

Design

Implementation

Test

Practice UDTs

Tutorial Summary

Polynomial UDT



Design of Polynomial UDT

Tutorial T08

Partha Pratim Das

Tutorial Recap

Objective & Outline

Int<N> UDT

Design

Operations

Class Design

Implementation

Test

Polynomial UDT

Design

Implementation

Test

Practice UDTs

Tutorial Summary

- Polynomials $A(x)$ of x having degree $\text{degree}(A) = n$ and $n + 1$ coefficients $a_0, a_1, a_2, \dots, a_n$:

$$A(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n = \sum_{i=0}^n a_i x^i$$

- The representation of a polynomial UDT `Poly` would need
 - a vector to keep the coefficients, and
 - a simple member to the degree (for null polynomials without coefficients)
- The types of coefficient and variable should be appropriate so that they can be multiplied and added. For simplicity, let us assume that they have the same type:

```
template<typename T = int> // Type of Coefficients and value
class Poly {               // a polynomial of type T
    vector<T> coeff_;      // coefficients
    size_t    deg_;       // deg_ = coeff_.size()-1
    // ... Rest of the class
}
```



Design of Polynomial UDT

Tutorial T08

Partha Pratim Das

Tutorial Recap

Objective & Outline

Int<N> UDT

Design

Operations

Class Design

Implementation

Test

Polynomial UDT

Design

Implementation

Test

Practice UDTs

Tutorial Summary

- A polynomial $A(x)$ of degrees n may be negated to generate polynomial $R(x)$ of degrees n by flipping the sign of every coefficient. That is:

$$R(x) = -A(x) = -\sum_{i=0}^n a_i x^i = \sum_{i=0}^n (-a_i) x^i$$

Hence,

$$r_i = -a_i, \quad 0 \leq i \leq n$$

- Two polynomials $A(x)$ and $B(x)$ of degrees n and m respectively may be added to generate polynomial $R(x)$ of degree $\max(n, m)$ by pairwise adding the coefficients of the same power. That is, for $n \geq m$

$$R(x) = A(x) + B(x) = \sum_{i=0}^n a_i x^i + \sum_{i=0}^m b_i x^i = \sum_{i=0}^m (a_i + b_i) x^i + \sum_{i=m+1}^n a_i x^i$$

Hence,

$$\begin{aligned} r_i &= a_i + b_i, \quad 0 \leq i \leq m \\ &= a_i, \quad m+1 \leq i \leq n \end{aligned}$$

Note: $A(x) - B(x) = A(x) + (-B(x))$



Design of Poly<T> Interface and Implementation

Tutorial T08

Partha Pratim Das

Tutorial Recap

Objective & Outline

Int<N> UDT

Design

Operations

Class Design

Implementation

Test

Polynomial UDT

Design

Implementation

Test

Practice UDTs

Tutorial Summary

```
template<typename T = int>           // Type of Coefficients and Value
class Poly { public:
    Poly(vector<T>& c = vector<T>( 1 )) : coeff_(c), deg_(c.size() - 1) { }
    Poly(size_t n) : coeff_(vector<T>(n+1)), deg_(n) { } // null polynomial
    Poly(const Poly& p) : coeff_(p.coeff_), deg_(p.deg_) { }
    ~Poly() { } // No virtual destructor needed
    Poly& operator=(const Poly& p) { deg_ = p.deg_; coeff_ = p.coeff_; return *this; }

    Poly operator+() const { return *this; }
    Poly operator-() const { Poly r(deg_);
        for (unsigned int i = 0; i <= deg_; i++) r.coeff_[i] = -coeff_[i];
        return r;
    }
    Poly operator+(const Poly& p) const { Poly r(max(p.deg_, deg_)); // result
        vector<T> v;
        if (deg_ > p.deg_) { v = p.coeff_; r.coeff_ = coeff_; } // copy the longer (shorter) vector
        else { v = coeff_; r.coeff_ = p.coeff_; } // of coefficients to r.coeff_ (v)
        for (unsigned int i = 0; i <= min(p.deg_, deg_); i++) { // add the common coefficients
            r.coeff_[i] = t.coeff_[i] + v[i];
        }
        return r;
    }
    Poly operator-(const Poly& p) const { return *this + (-p); }
```



Design of Poly<T> Interface and Implementation

Tutorial T08

Partha Pratim Das

Tutorial Recap

Objective & Outline

Int<N> UDT

Design

Operations

Class Design

Implementation

Test

Polynomial UDT

Design

Implementation

Test

Practice UDTs

Tutorial Summary

```
Poly& operator+=(const Poly& p) { *this = *this + p; return *this; }
Poly& operator-=(const Poly&p) { *this = *this - p; return *this; }

friend ostream& operator<<(ostream& os, const Poly& p) { int j;
    for (j = p.deg_; j >= 0; --j)
        { if (static_cast<T>(0) != p.coeff_[j]) break; } // first non-zero coeff.
    if (0 > j)
        os << 0;
    else
        if (0 == j) os << p.coeff_[j];
        else os << p.coeff_[j] << "x^" << j;
    for (int i = j-1; i >= 0; --i) {
        if (static_cast<T>(0) != p.coeff_[i]) {
            if (0 != i)
                if (static_cast<T>(1) != p.coeff_[i])
                    os << " + " << p.coeff_[i] << "x^" << i;
                else
                    os << " + " << "x^" << i;
            else
                os << " + " << p.coeff_[i];
        }
    }
    os << ".";
    return os;
}
```



Design of Poly<T> Interface and Implementation

Tutorial T08

Partha Pratim Das

Tutorial Recap

Objective & Outline

Int<N> UDT

Design

Operations

Class Design

Implementation

Test

Polynomial UDT

Design

Implementation

Test

Practice UDTs

Tutorial Summary

```
friend istream& operator >>(istream& is, Poly& p) {
    cout << "Enter degree of the polynomial ";
    is >> p.deg_;
    p.coeff_.resize(p.deg_ + 1);
    cout << "Enter all the coefficients like a0+a1*x+a2*x^2+....an*x^n" << endl;
    for (unsigned int i = 0; i <= p.deg_; i++)
        is >> p.coeff_[i];
    return is;
}

// Evaluates the polynomial - use Horner's Rule
T operator()(const T& x) {
    T val = 0;
    for (int i = deg_; i >= 0; i--)
        val = val * x + coeff_[i];
    return val;
}

private:
    vector<T>    coeff_;
    size_t      deg_;
    template<typename T> inline static T max(T a, T b) { return a > b ? a : b; }
    template<typename T> inline static T min(T a, T b) { return a < b ? a : b; }
};
```



Testing Poly<T>

Tutorial T08

Partha Pratim Das

Tutorial Recap

Objective & Outline

Int<N> UDT

Design

Operations

Class Design

Implementation

Test

Polynomial UDT

Design

Implementation

Test

Practice UDTs

Tutorial Summary

```

#include <iostream>
using namespace std;
#include "fraction.h"
#include "polynomial.h"

void main() { vector<int> v = { 1, 2, 1 };
    Poly<int> p(v); cout << "p(x): " << p << " p(2) = " << p(2); // p(x): 1x^2 + 2x^1 + 1. p(2) = 9
    Poly<int> q(p); cout << "q(p): " << q << " q(2) = " << q(2); // q(p): 1x^2 + 2x^1 + 1. q(2) = 9
    Poly<int> s(vector<int>({ 0, 0, 1, 2, 1, 0, 2, 7, 0 }));
    cout << "s(x): " << s << " s(1) = " << s(1); // s(x): 7x^7 + 2x^6 + x^4 + 2x^3 + x^2. s(1) = 13
    Poly<int> r; cout << "r: " << r << " r(2) = " << r(2); // r: 1. r(2) = 1

    cin >> r; // 2 1 2 1
    cout << "r: " << r << " r(2) = " << r(2); // r: 1x^2 + 2x^1 + 1. r(2) = 9

    Poly<int> t(2); cout << "t(x): " << t << " t(2) = " << t(2); // t(x): 0. t(2) = 0

    r = p; cout << "r = p: " << r << " r(2) = " << r(2); // r = p: 1x^2 + 2x^1 + 1. r(2) = 9
    r = -p; cout << "r = -p: " << r << " r(2) = " << r(2); // r = -p: -1x^2 + -2x^1 + -1. r(2) = -9

    p = vector<int>({ 1, 5, 6 });
    cout << "p(x): " << p << " p(2) = " << p(2); // p(x): 6x^2 + 5x^1 + 1. p(2) = 35

    q = vector<int>({ 1, -2, 1 });
    cout << "q(x): " << q << " q(2) = " << q(2); // q(x): 1x^2 + -2x^1 + 1. q(2) = 1

```



Testing Poly<T>

Tutorial T08

Partha Pratim Das

Tutorial Recap

Objective &
Outline

Int<N> UDT

Design

Operations

Class Design

Implementation

Test

Polynomial UDT

Design

Implementation

Test

Practice UDTs

Tutorial Summary

```
r = p + q; cout << "r = p + q: " << r << " r(2) = " << r(2); // r = p + q: 7x^2 + 3x^1 + 2. r(2) = 36
r = p - q; cout << "r = p - q: " << r << " r(2) = " << r(2); // r = p - q: 5x^2 + 7x^1. r(2) = 34
r = p; p += q; cout << "p += : " << p << " p(2) = " << p(2); // p += : 7x^2 + 3x^1 + 2. p(2) = 36
p = r; p -= q; cout << "p -= : " << p << " p(2) = " << p(2); // p -= : 5x^2 + 7x^1. p(2) = 34
```

```
vector<Fraction> vf = { Fraction(1, 2), Fraction(-3, 5), Fraction(2, 3) };
Poly<Fraction> pf1(vf); cout << "pf1(x): " << pf1 << " pf1(2) = " << pf1(2) << endl;
// pf1(x): 2/3x^2 + -3/5x^1 + 1/2. pf1(2) = 59/30
```

```
Poly<Fraction> pf2; cout << "pf2(x): " << pf2 << " pf2(2) = " << pf2(2) << endl;
// pf2(x): 1. pf2(2) = 1
```

```
cin >> pf2; // 1 2 3 1 2
cout << "pf2: " << pf2 << " pf2(2) = " << pf2(2) << endl;
// pf2(x): 1/2x^1 + 2/3. pf2(2) = 5/3
```

```
Poly<Fraction> pf3 = pf1 + pf2;
cout << "pf3(x): " << pf3 << " pf3(2) = " << pf3(2) << endl;
// pf3(x): 2/3x^2 + -1/10x^1 + 7/6. pf3(2) = 109/30
```

```
Poly<Fraction> pf4 = pf1 - pf2;
cout << "pf4(x): " << pf4 << " pf4(2) = " << pf4(2) << endl;
// pf4(x): 2/3x^2 + -11/10x^1 + -1/6. pf4(2) = 3/10
```

```
}
```



Practice UDTs

Tutorial T08

Partha Pratim
Das

Tutorial Recap

Objective &
Outline

Int<N> UDT

Design

Operations

Class Design

Implementation

Test

Polynomial UDT

Design

Implementation

Test

Practice UDTs

Tutorial Summary

Practice UDTs



Practice UDTs

Tutorial T08

Partha Pratim Das

Tutorial Recap

Objective & Outline

Int<N> UDT

Design

Operations

Class Design

Implementation

Test

Polynomial UDT

Design

Implementation

Test

Practice UDTs

Tutorial Summary

- **Fraction**
 - Change binary arithmetic and comparison operators to **friend** functions from methods
 - Parameterize **Fraction** with type **T = int**
 - Provide mixed mode support
- **Int<N>**
 - Implement **operator*()**
 - Implement **operator/()**
 - Implement **operator%()**
- **Poly<T>**
 - Test **Poly<double>**
 - Use member function template for **operator()()** with another type parameter **U** for **x**
 - Analyze the compatibility issues between types **T** and **U**
- **Mixed UDTs**
 - Test **Fraction<Int<N> >**
 - Test **Poly<Int<N> >**
 - Test **Poly<Fraction<Int<N> > >**



Tutorial Summary

Tutorial T08

Partha Pratim Das

Tutorial Recap

Objective & Outline

`Int<N>` UDT

Design

Operations

Class Design

Implementation

Test

Polynomial UDT

Design

Implementation

Test

Practice UDTs

Tutorial Summary

- Presented the design, implementation and test for `Int<N>` and `Poly<T>` types
- Showed how `Poly<int>` as well as `Poly<Fraction>` works
- Outlined several practice UDTs for homework